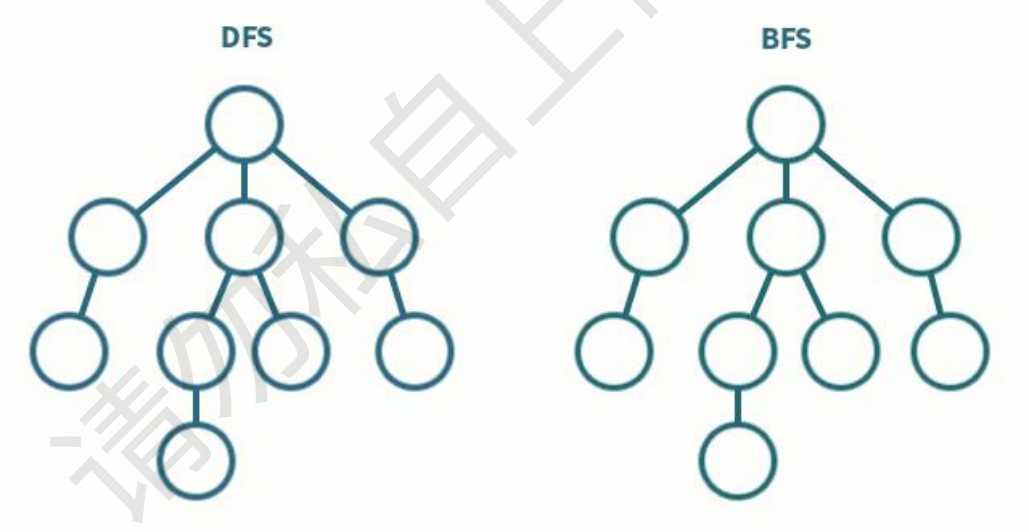
搜索求解策略



计算机与人工智能学院
马冲

请勿私自上传课件

搜索求解策略

1.1 搜索的概念

1.2 盲目搜索

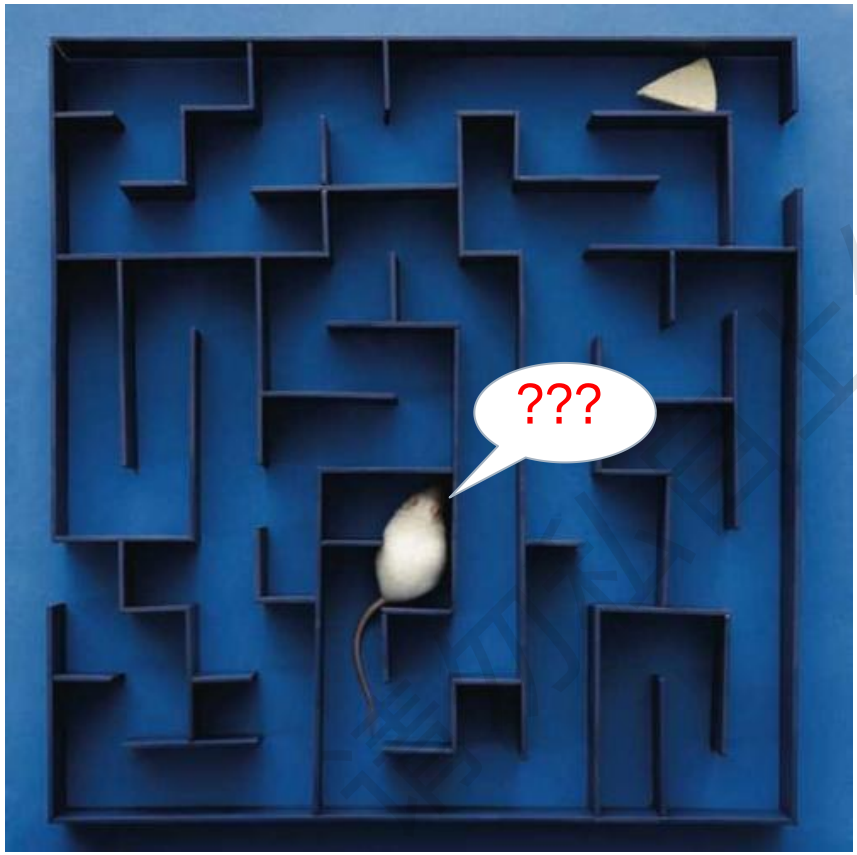
1.3 启发式搜索

1.4 博弈树搜索

请勿私自上传本课件

1.1 搜索的概念

基本概念



求解一个问题时，涉及两个方面：

一是该问题的表示，如果一个问题找不到一个合适的表示方法，就谈不上对它求解。

另一方面则是选择一种相对合适的求解方法。由于绝大多数需要人工智能方法求解的问题缺乏直接求解的方法，而搜索可以作为一种求解问题的通用策略。

清华大学出版社 上传课件

1.1 搜索的概念

基本概念

定义： 根据问题的实际情况，利用已有知识，构造一条代价较小的推理路径，使问题得到解决的过程。

本质： 在**状态空间**中寻找从初始状态到目标状态的一条有效路径。

搜索中需要解决的核心问题

问题	说明
(1) 是否一定能找到解?	**完备性**（Completeness）
(2) 找到的是否是最优解?	**最优性**（Optimality）
(3) 时间与空间复杂度如何?	效率问题
(4) 是否会陷入死循环?	需避免重复状态

请勿私自上传本课件

1.1 搜索的概念

搜索策略

按照搜索方向划分：

- **正向搜索（数据驱动）：** 从初始状态出发 → 目标（如广度优先搜索BFS、深度优先搜索DFS）
- **逆向搜索（目的驱动）：** 从目标状态反推 → 初始
- **双向搜索：** 两端同时搜索，中间汇合（效率更高）

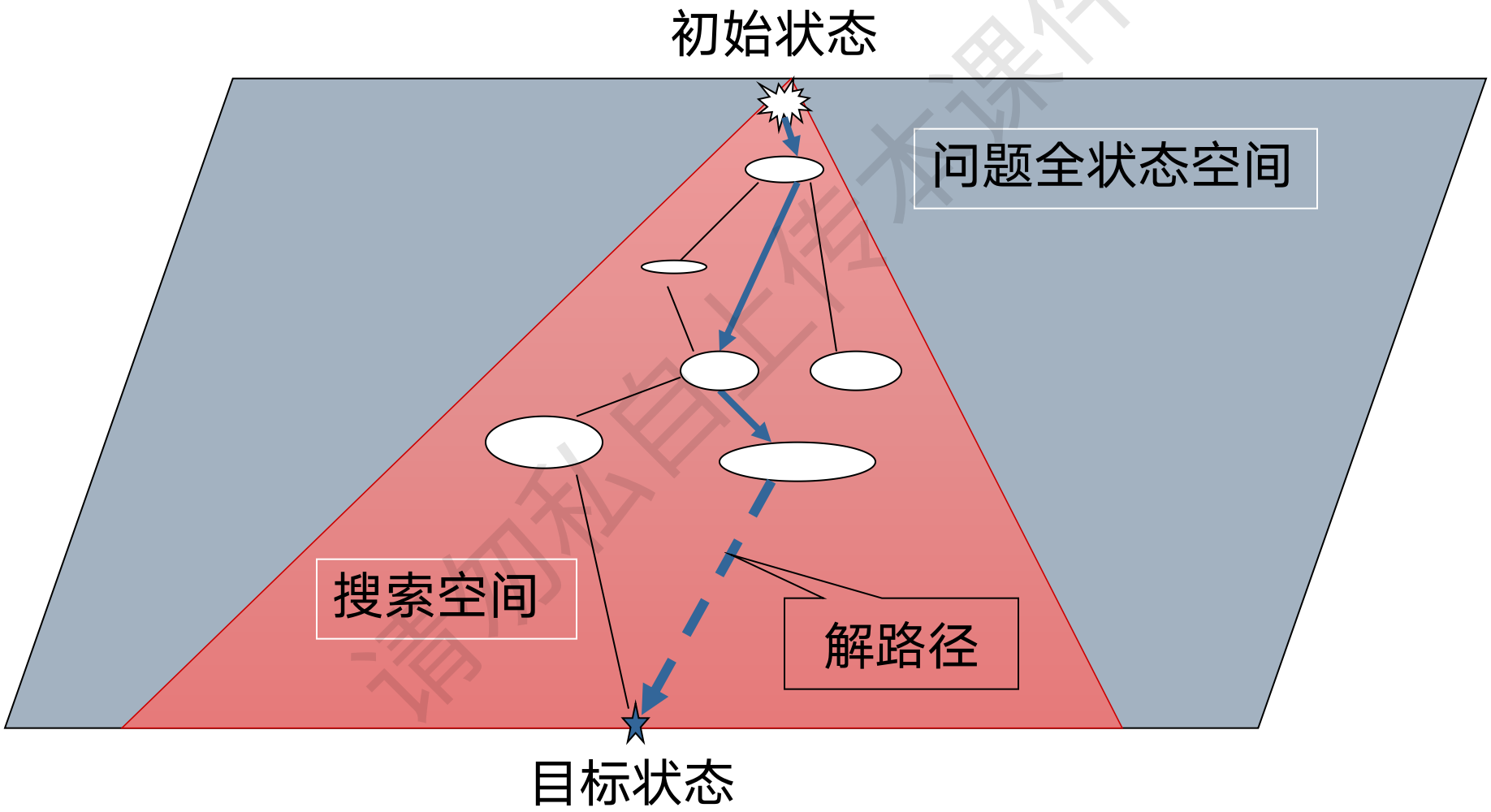
按照信息运用方式划分：

- **盲目搜索：** 不利用问题领域知识，按固定策略遍历状态空间
- **启发式搜索：** 利用问题领域知识（启发信息）引导搜索，减少盲目扩展。

请勿私自上传课件

1.1 搜索的概念

搜索状态空间



请勿私自传播课件

1.1 搜索的概念

状态空间的知识表示

- **状态：**表示系统状态、事实等叙述型知识的一组变量或数组；
- **操作：**表示引起状态变化的过程型知识的一组关系或函数；
- **状态空间：**利用状态变量和操作符号，表示系统或问题的有关知识的符号体系，状态空间可用四元组表示

$$(S, O, S_0, G)$$

S ：状态集合。

O ：操作算子的集合。

S_0 ：包含问题的初始状态（S 的非空子集）。

G ：包含问题的目的状态（S 的非空子集）。

请勿私自上传本课件

1.1 搜索的概念

状态空间的知识表示

- **求解路径**：从 S_0 节点到 G 节点的路径。
- **状态空间的一个解**：一个有限的操作算子序列。

$$S_0 \xrightarrow{O_1} S_1 \xrightarrow{O_2} S_2 \xrightarrow{O_3} \cdots \xrightarrow{O_k} G$$

$O_1, \cdots, O_k$ ：状态空间的一个解。

实例：八数码问题的状态空间

2	3	1
5		8
4	6	7

初始状态

1	2	3
8		4
7	6	5

目标状态

状态集：

所有摆法

操作算子：

将空格向上移Up

将空格向左移Left

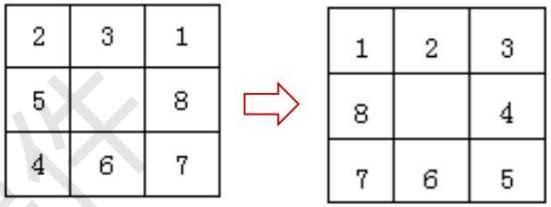
将空格向下移Down

将空格向右移Right

请勿私自上传本课件

1.1 搜索的概念

■ 状态空间的知识表示



初始状态　　目标状态

➢ 什么是八数码问题？

八数码问题的核心是：在一个受限的操作规则下，通过一系列合法动作，从初始状态转移到目标状态。

这种“状态转移 + 约束操作 + 目标导向”的结构，正是大量现实问题的共性模型。

➢ 应用领域？

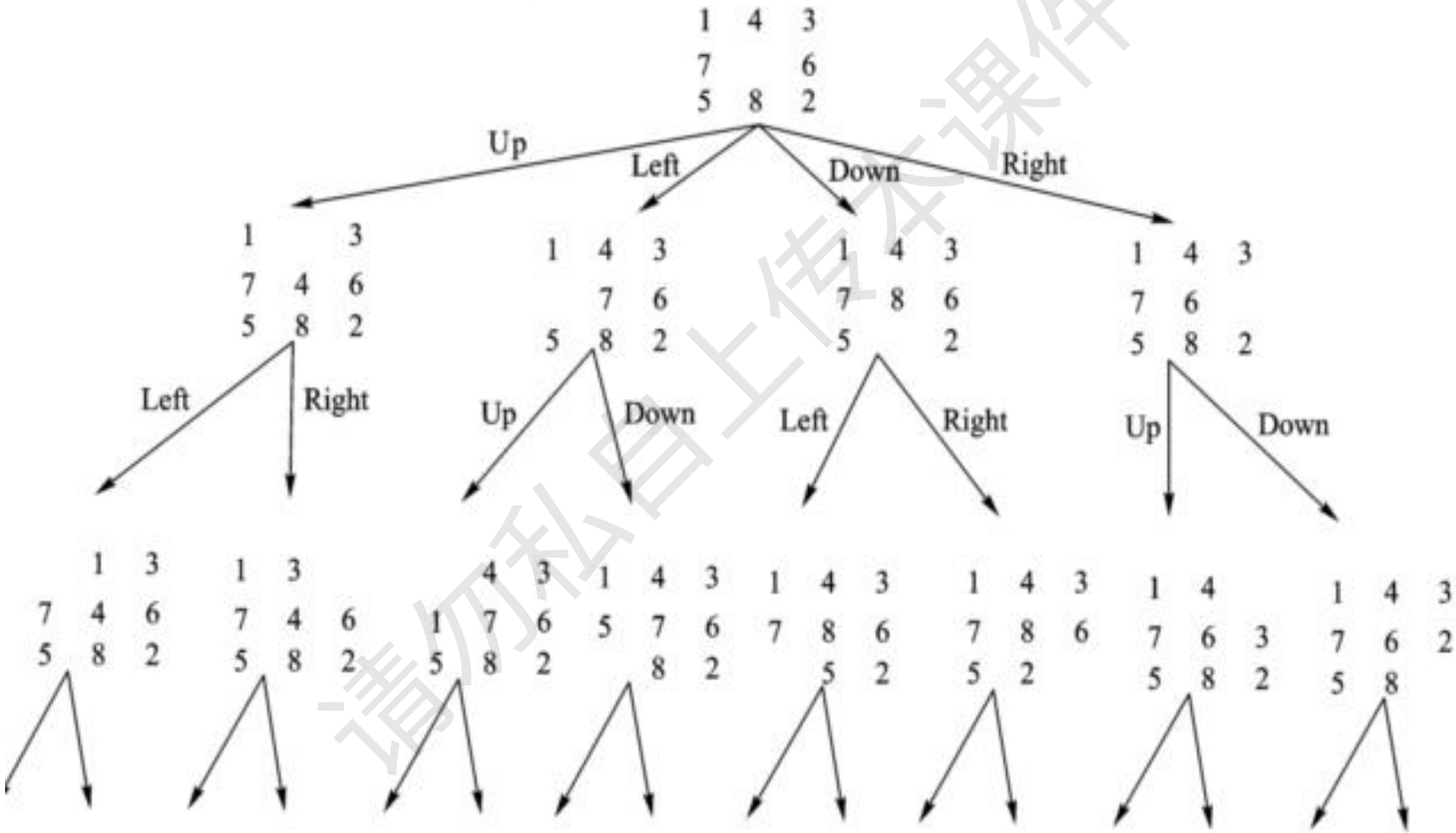
- 机器人路径规划与运动控制（移动、避障）
- 自动仓储与物流调度（货箱自动快速出库）
- 集成电路布局与布线
- 拼图与图像修复

请勿私自上传本课件

1.1 搜索的概念

■ 状态空间的知识表示

八数码问题



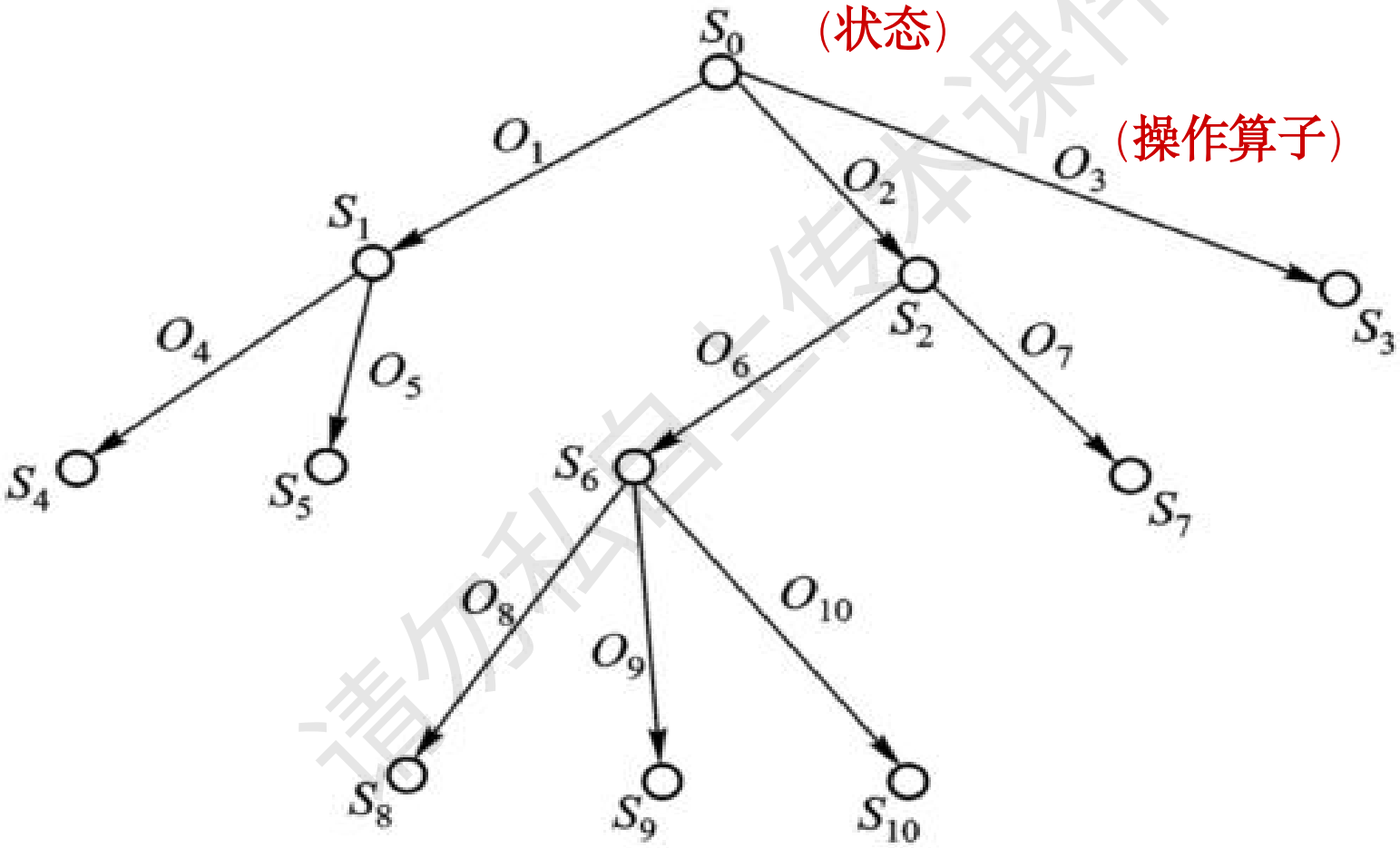
八数码状态空间图

请勿私自上传本课件

1.1 搜索的概念

■ 状态空间的知识表示

八数码问题



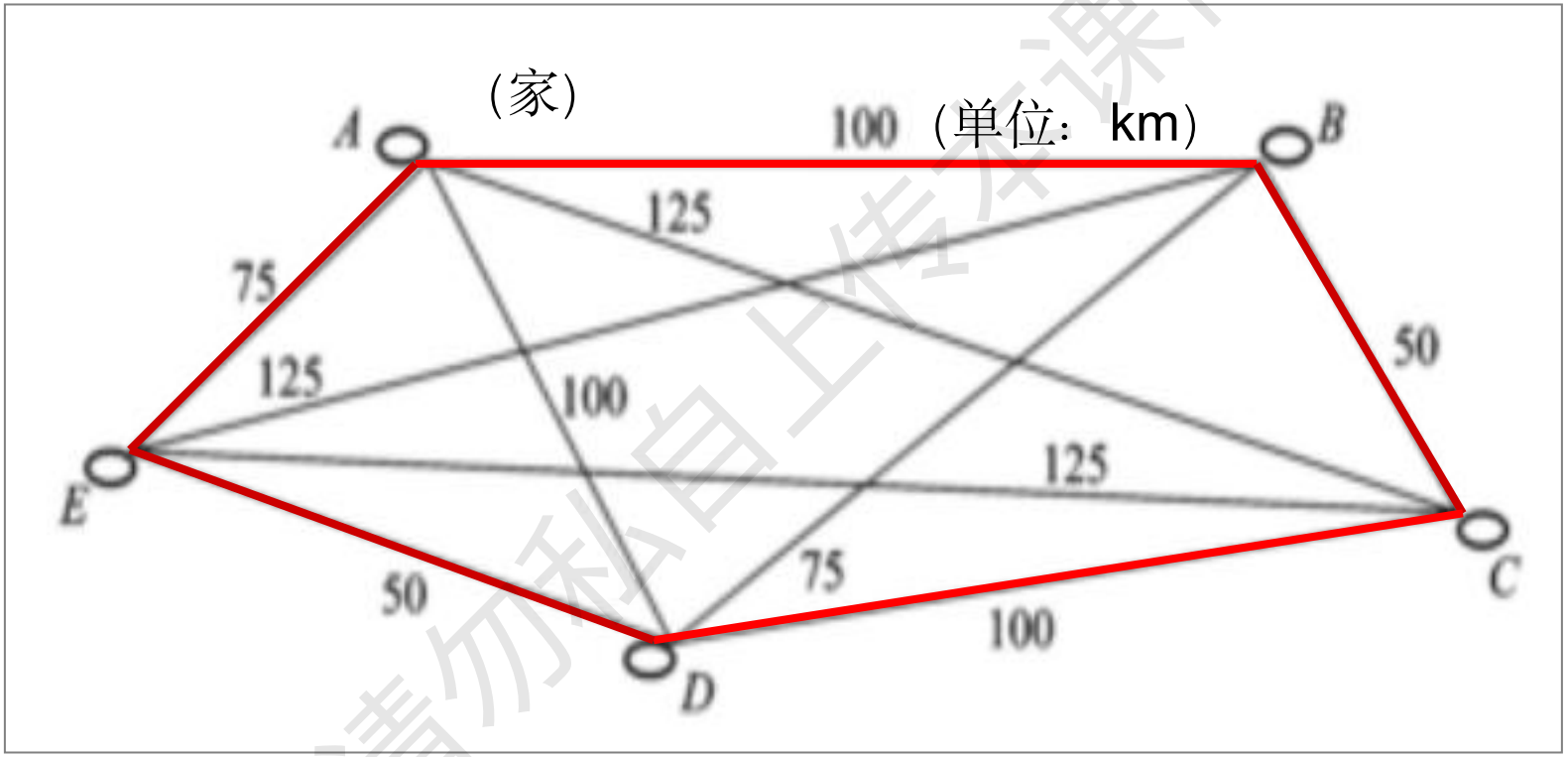
八数码状态空间的有向图描述

请勿私自上传本课件

1.1 搜索的概念

■ 状态空间的知识表示

TSP问题



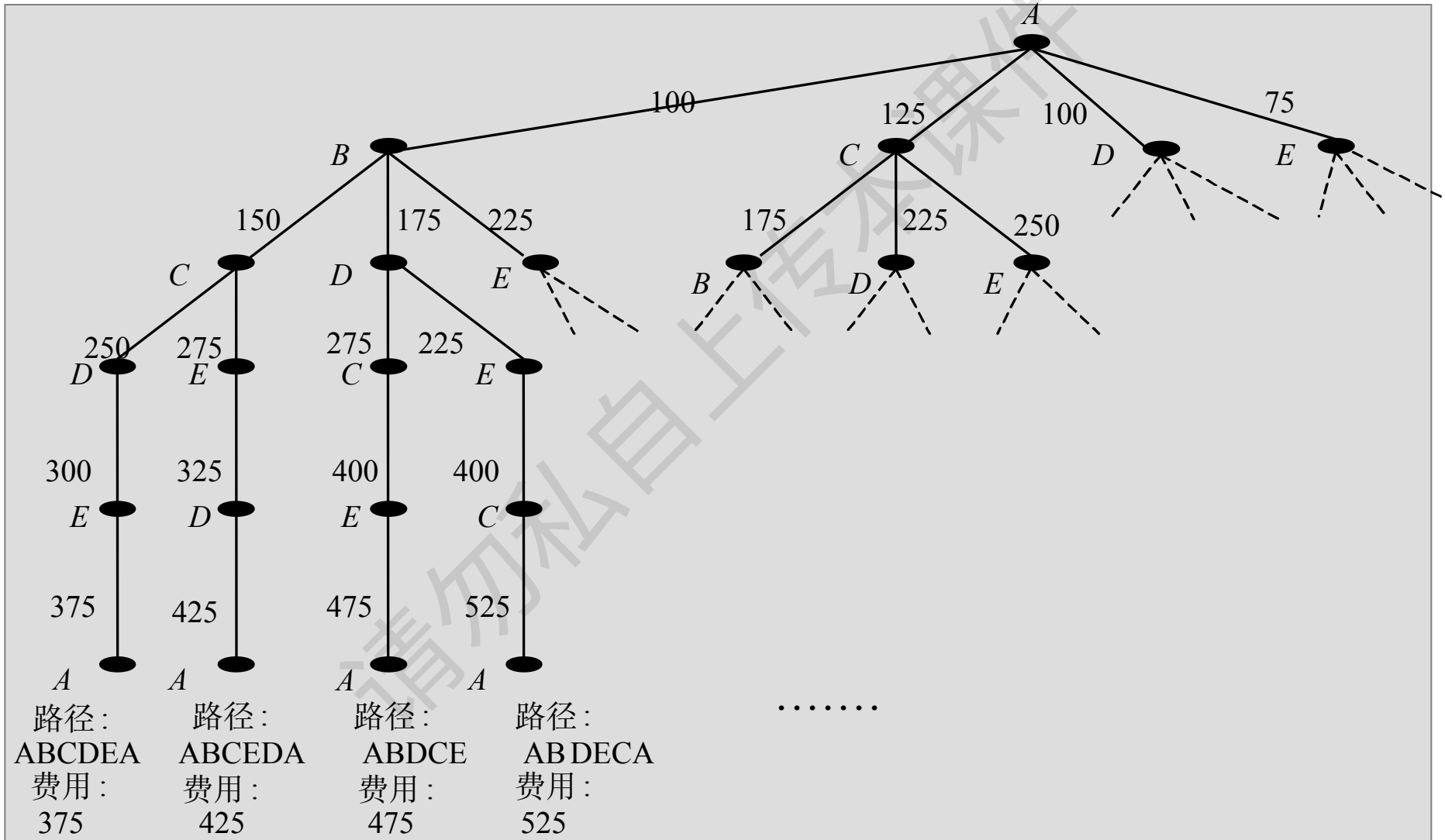
可能路径：费用为375的路径（A，B，C，D，E，A）

请勿私自上传本课件

1.1 搜索的概念

■ 状态空间的知识表示

TSP问题



旅行推销员状态空间图（部分）

请勿私自上传本课件

1.1 搜索的概念

■ 图搜索通用框架（OPEN/CLOSED表）

OPEN表

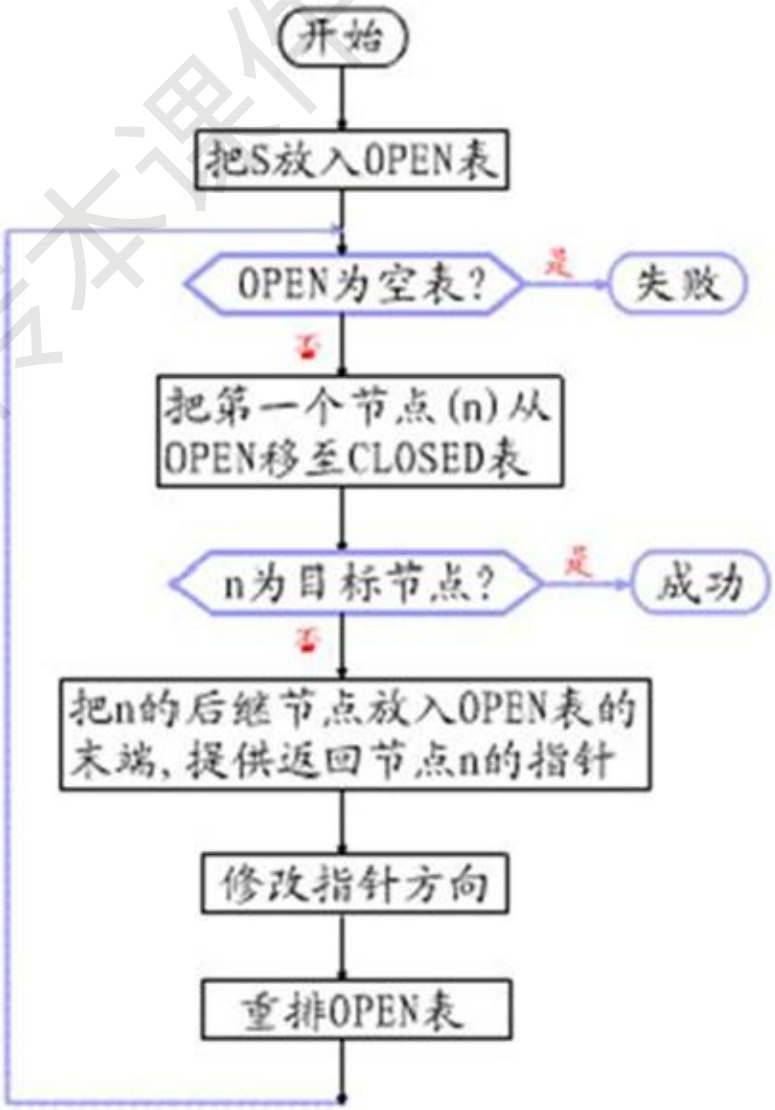
节点	父节点

CLOSED表

编号	节点	父节点

关键步骤：

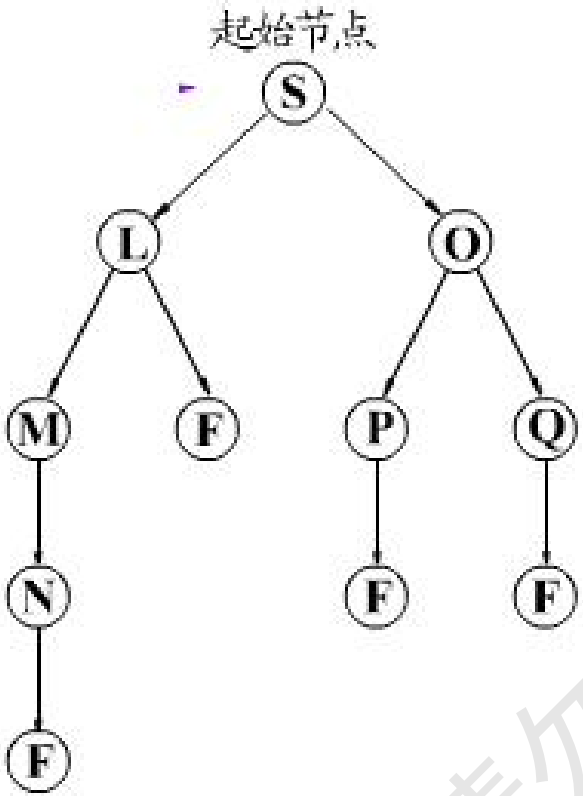
- 将初始节点加入 OPEN；
- 若 OPEN 为空 → 失败；
- 取出一个节点 n，放入 CLOSED；
- 若 n 是目标 → 成功，回溯路径；
- 若不是，扩展 n，生成子节点，过滤已在 OPEN/CLOSED 中的节点；
- 将新子节点加入 OPEN，转步骤 2。

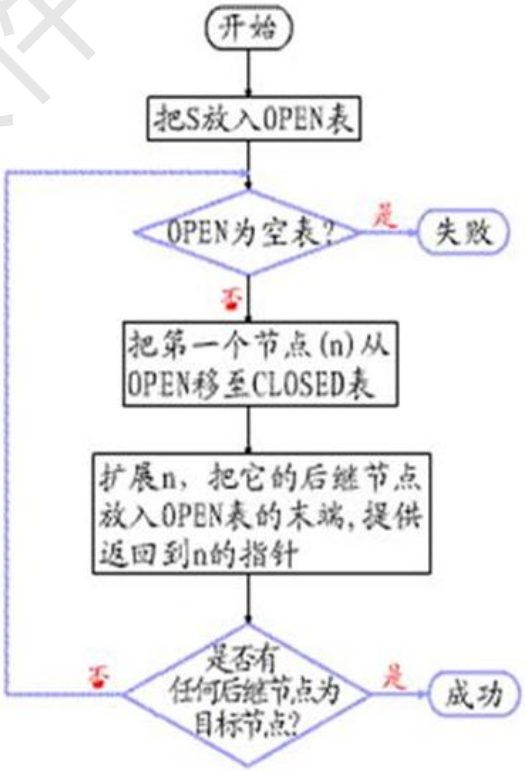


请勿私自上传本课件

1.1 搜索的概念

■ 图搜索通用框架（OPEN/CLOSED表）





- open表：已经生成出来但其子状态未被搜索的状态。
- closed表：记录了已被生成扩展过的状态。

请勿私自上传本课件

1.1 搜索的概念

■ 状态空间的图搜索策略

- 状态空间搜索策略
  - 盲目搜索
    - 广度优先搜索
    - 深度优先搜索
    - 有界深度优先搜索
    - 代价树的广度优先搜索
    - 代价树的深度优先搜索
  - 启发式搜索
    - 局部择优搜索
    - 全局择优搜索
    - A*算法

✓**盲目搜索**：在不具有对特定问题的任何有关信息的条件下，按固定的步骤（依次或随机调用操作算子）进行的搜索。

✓**启发式搜索**：考虑特定问题领域可应用的知识，动态地确定调用操作算子的步骤，优先选择较适合的操作算子，尽量减少不必要的搜索，以求尽快地到达结束状态。

请勿私自上传课件

搜索求解策略

1.1 搜索的概念

1.2 盲目搜索

1.3 启发式搜索

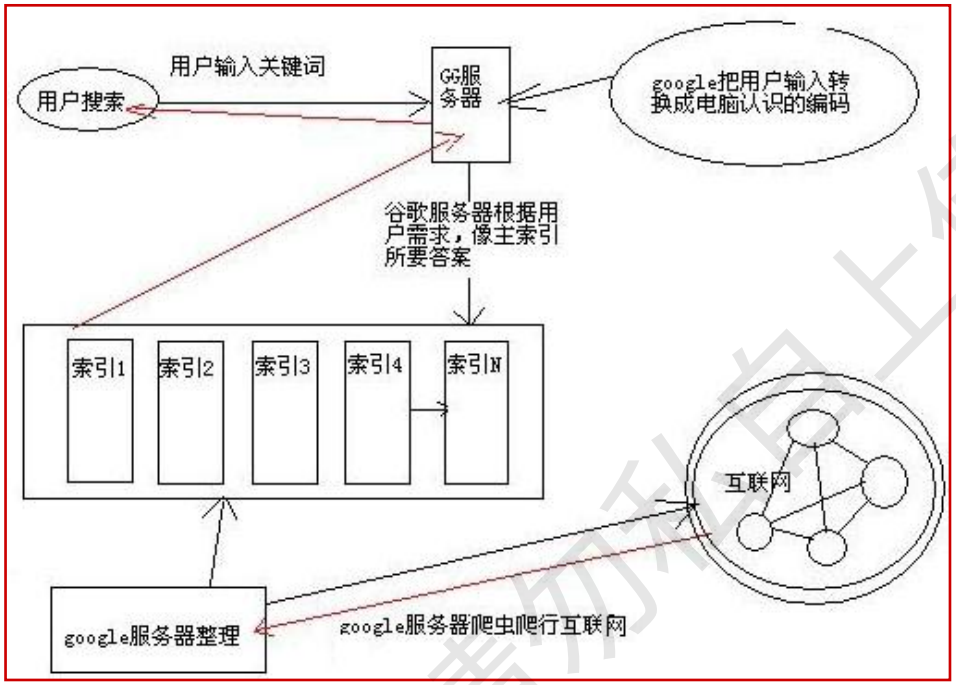
1.4 博弈树搜索

请勿私自上传本课件

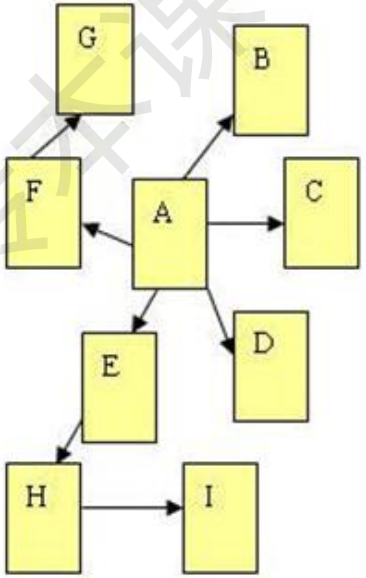
1.2 盲目搜索

深度优先与宽度优先

什么是深度优先与宽度优先？以搜索引擎中的互联网爬虫为例

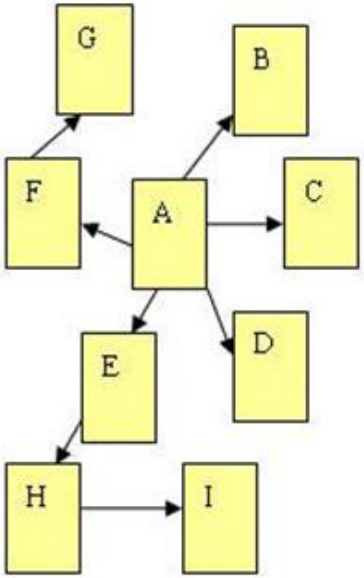


网络爬虫示例



广度优先的抓取顺序：
A—B.C.D.E.F—H.G—I

宽度优先



深度优先的抓取顺序：
A—F—G
E—H—I

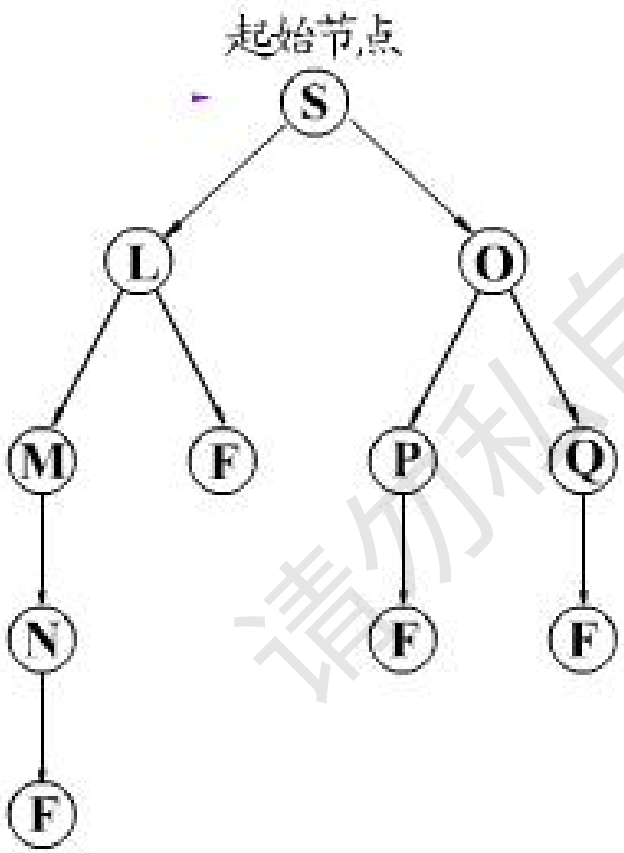
深度优先

请勿私自上传本课件

1.2 盲目搜索

■ 宽度优先搜索 BFS

定义：如果搜索是以接近起始节点的程度依次扩展节点的，那么这种搜索就叫做宽度优先搜索(breadth-first search)。



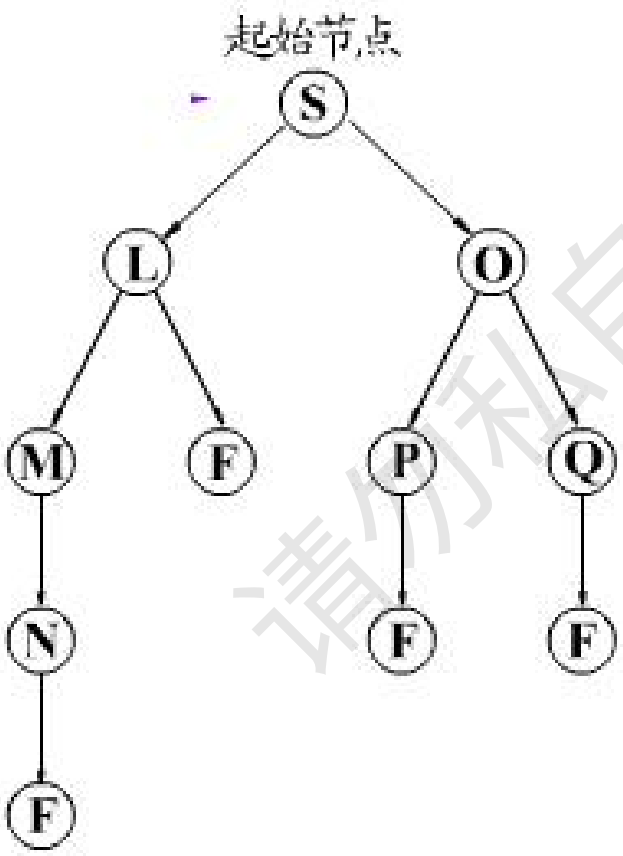
从图可见，这种搜索是逐层进行的；在对下一层的任一节点进行搜索之前，必须搜索完本层的所有节点。

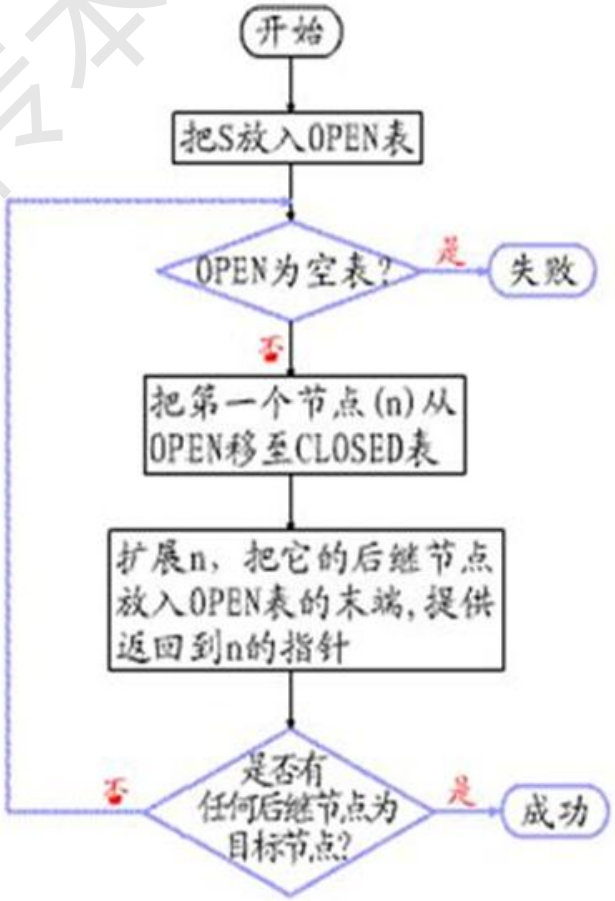
请勿私自上传本课件

1.2 盲目搜索

■ 宽度优先搜索 BFS

- 策略：先扩展深度最浅的节点（逐层扩展）；
- open表的结构为队列（先进先出）





请勿私自上传本课件

1.2 盲目搜索

■ 宽度优先搜索 BFS

宽度优先搜索方法能够保证在搜索树中找到一条通向目标节点的最短途径；这棵搜索树提供了所有存在的路径(如果没有路径存在，那么对有限图来说，我们就说该法失败退出；对于无限图来说，则永远不会终止)。

性质：

完备：若有解，必能找到；

最优（当所有操作代价相等时）；

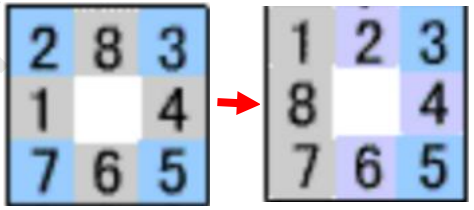
空间复杂度高：$O(b^d)$，b=分支因子，d=解深度。

请勿私自上传本课件

1.2 盲目搜索

■ 宽度优先搜索 BFS

例：宽度优先搜索应用于八数码难题

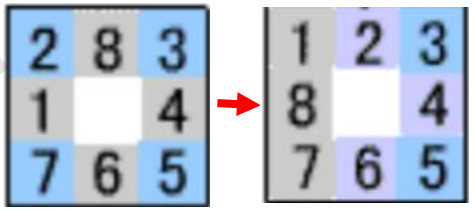


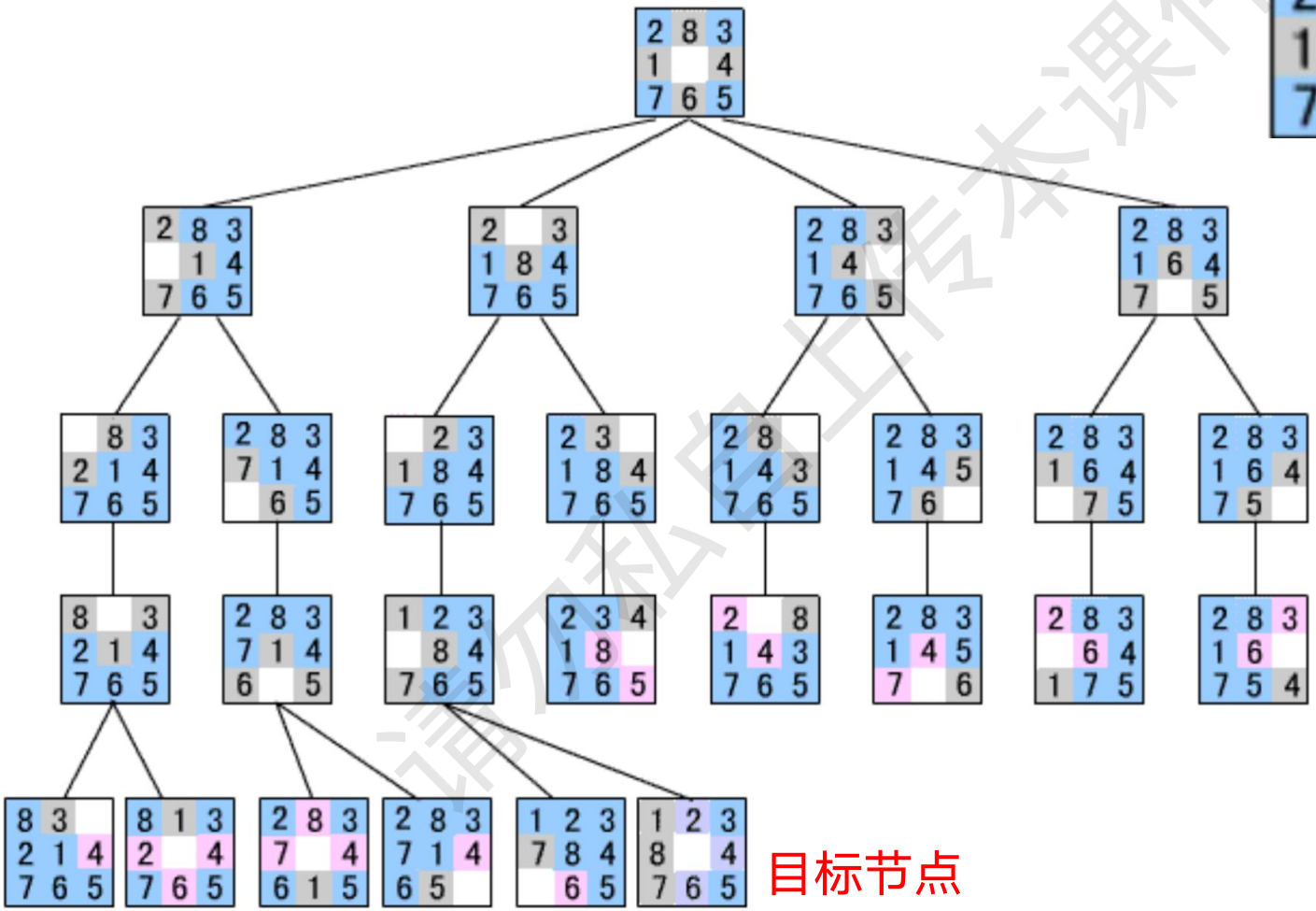
请勿私自上传本课件

1.2 盲目搜索

■ 宽度优先搜索 BFS

例：宽度优先搜索应用于八数码难题





目标节点

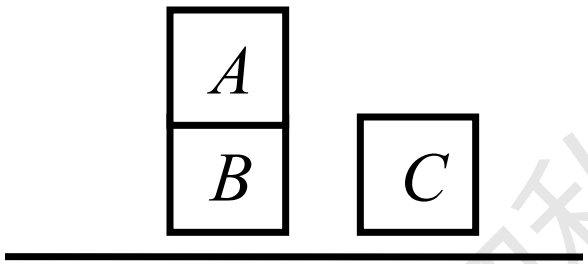
请勿私自传播本课件

1.2 盲目搜索

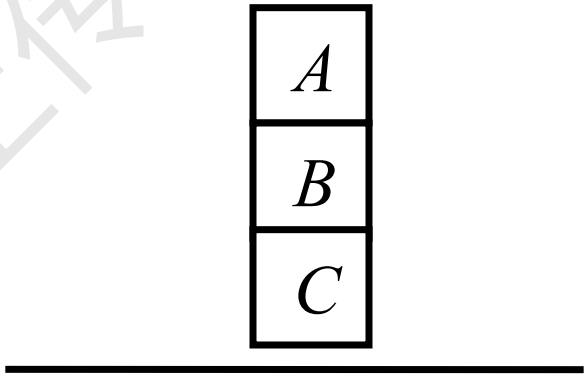
■ 宽度优先搜索

例：积木问题

例 通过搬动积木块，希望从初始状态达到一个目的状态，即三块积木堆叠在一起。



(a) 初始状态



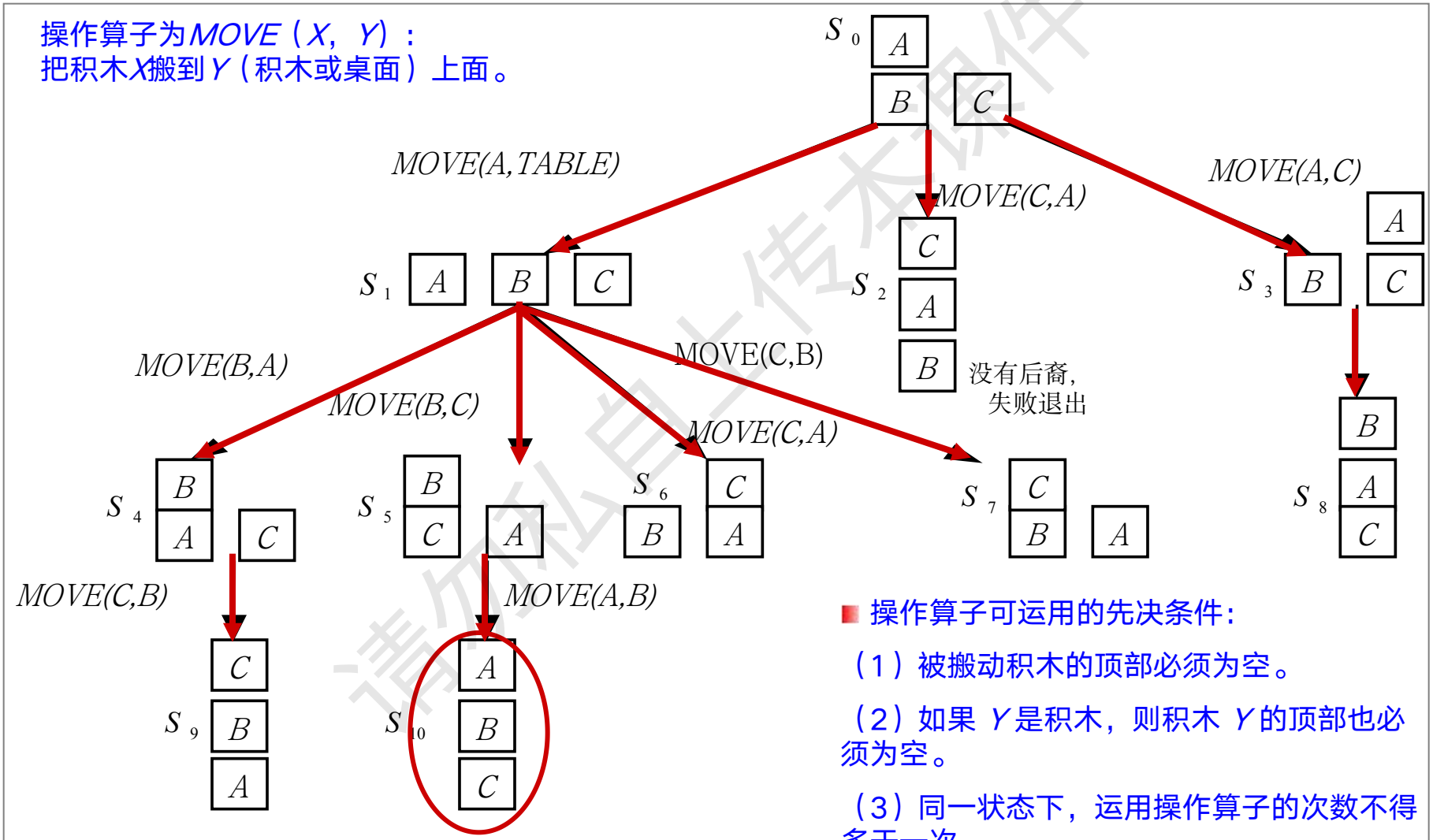
(b) 目的状态

请勿私自上传本课件

1.2 盲目搜索

■ 宽度优先搜索

操作算子为*MOVE*（*X*，*Y*）：
把积木*X*搬到*Y*（积木或桌面）上面。



■ 操作算子可运用的先决条件：

（1）被搬动积木的顶部必须为空。

（2）如果 *Y*是积木，则积木 *Y*的顶部也必须为空。

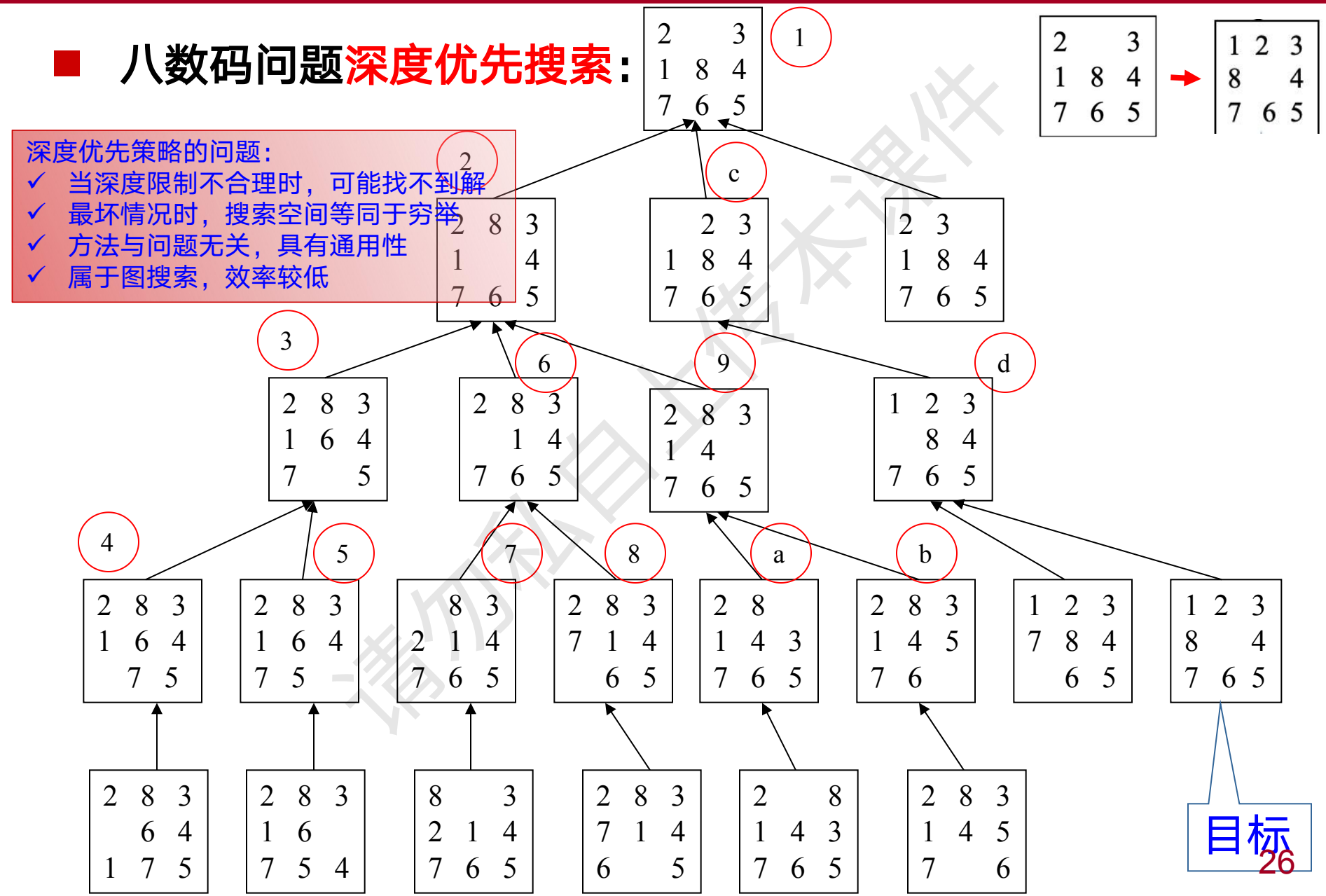
（3）同一状态下，运用操作算子的次数不得多于一次。

积木问题的宽度优先搜索树

请勿私自上传本课件

1.2 盲目搜索

■ 八数码问题深度优先搜索：



请勿私自上传本课件

1.2 盲目搜索

■ 深度优先搜索 DFS

定义：在所搜过程中，首先扩展最新产生的(即最深的)节点的方式称为深度优先搜索(depth-first search)。深度相等的节点可以任意排列。

性质：

- 不完备（可能陷入无限分支）
- 不保证最优
- 空间复杂度低：$O(bm)$，m=最大深度（可剪枝）

改进：深度限制搜索（DLS） + 迭代加深搜索（IDS）

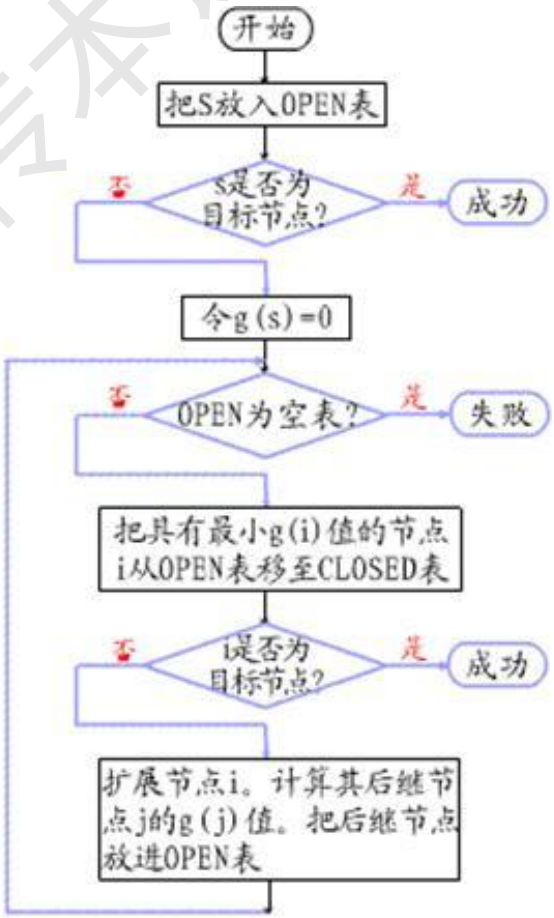
- IDS = 以深度 0,1,2,… 重复运行 DLS;
- 兼具 BFS 的完备/最优性 + DFS 的低空间消耗!

请勿私自上传本课件

1.2 盲目搜索

■ 深度优先搜索 DFS

- 策略：先扩展最新生成（最深）的节点；
- open表的结构为栈（先进后出）



请勿私自上传本课件

搜索求解策略

1.1 搜索的概念

1.2 盲目搜索

1.3 启发式搜索

1.4 博弈树搜索

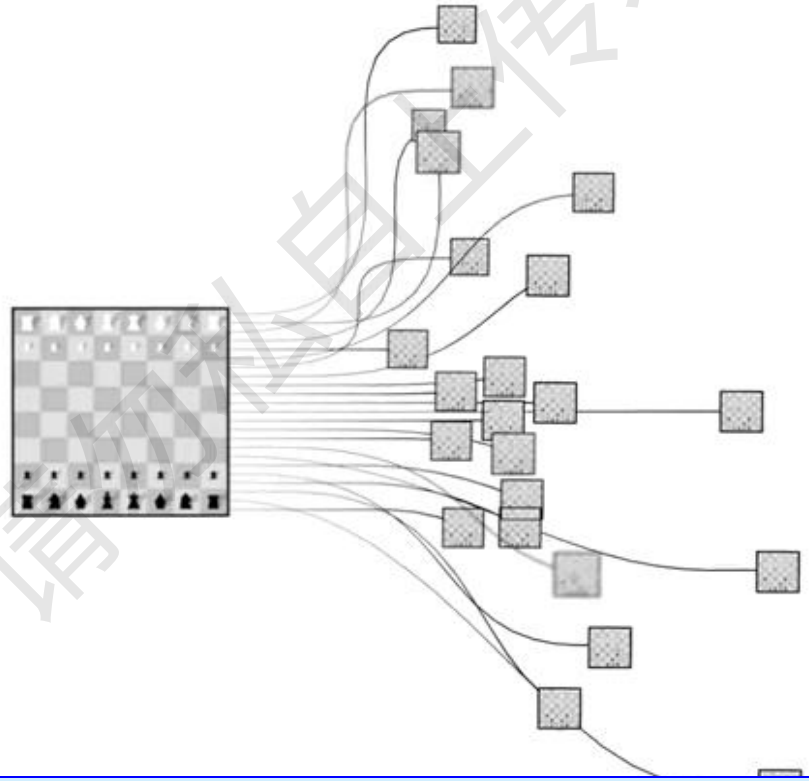
请勿私自上传本课件

1.3 启发式搜索

- **运用启发式策略的两种基本情况：**

✓ 由于在问题陈述和数据获取方面固有的模糊性，可能会使它**没有一个确定的解**。

✓ 虽然一个问题可能有确定解，但是其**状态空间特别大**，搜索中生成扩展的状态数会随着**搜索的深度呈指数级增长**。



一字棋：9！，西洋跳棋：10^{78}，国际象棋：10^{120}，围棋：10^{761}

请勿私自上传本课件

1.3 启发式搜索

- **盲目搜索的不足：效率低，耗费过多的计算空间与时间。**

通过分析宽度优先、深度优先搜索算法,其主要的差别是OPEN表中待扩展节点的顺序问题。怎么找到一种方法用于排列待扩展节点的顺序，即选择最有希望的节点加以扩展，搜索效率将会大为提高。

- **启发信息的引入：**搜索过程一般需要某些有关具体问题领域特性的、与具体问题求解过程有关的、并可指导搜索过程朝着最有希望方向前进的控制信息，我们把此种信息叫做启发信息。

启发信息一般可分为以下三类：

- （辅助选择节点）用于扩展节点的选择, 即决定应先扩展哪一个节点, 以免盲目扩展。
- （辅助选择操作算子）用于生成节点的选择,决定应生成哪些后续节点,以免盲目地生成过多无用节点。
- （辅助删除节点）用于删除节点的选择,即决定应删除哪些无用节点（剪枝）, 以免造成进一步的时空搜索浪费。

1.3 启发式搜索

- 把利用启发信息的搜索方法叫做启发式搜索方法
  - ✓ 在状态空间搜索中，**启发式**被定义成一系列操作算子，并能从状态空间中选择最有希望到达问题解的路径。
  - ✓ **启发式策略**：利用与问题有关的启发信息进行搜索。
- 核心思想

  利用问题领域知识（启发信息）引导搜索，减少盲目扩展。
- **本节课重点关注第一种启发信息的运用**
  - ➢用于扩展节点的选择，即决定应先扩展哪一个节点，以免盲目扩展。这种搜索总是选择“最有希望”的节点作为下一个被扩展的节点。

请勿私自上传本课件

1.3 启发式搜索

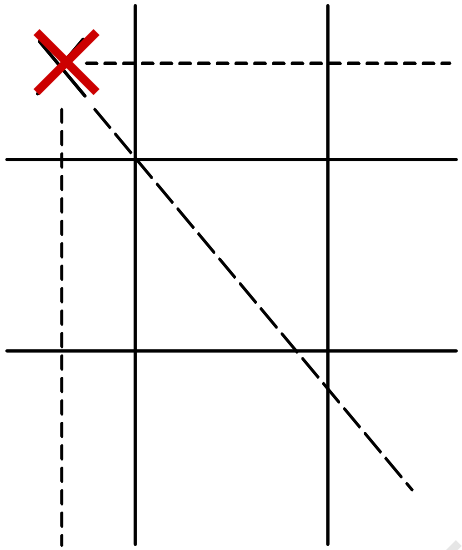
- **启发式搜索实例：一字棋**。在九宫棋盘上，从空棋盘开始，双方轮流在棋盘上摆各自的棋子 × 或 ○ （每次一枚），谁先取得三子一线（一行、一列或一条对角线）的结果就取胜。

  - ✓ × 和 ○ 能够在棋盘中摆成的各种不同的棋局就是问题空间中的不同状态。
  - ✓ 9个位置上摆放{空， ×， ○}有 3^9 种棋局。
  - ✓ 可能的走法：$9\times8\times7\times\cdots\times1$ 。

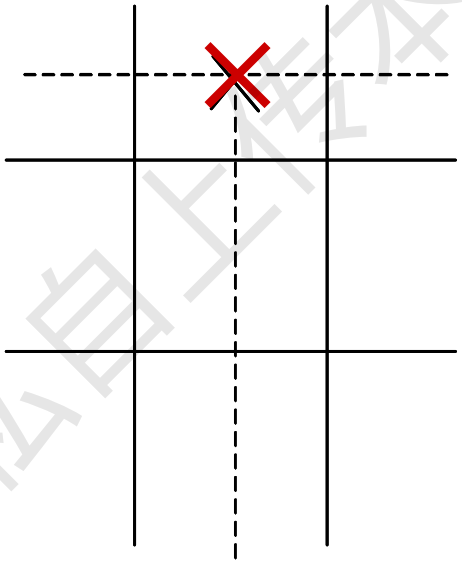
请勿私自上传本课件

1.3 启发式搜索

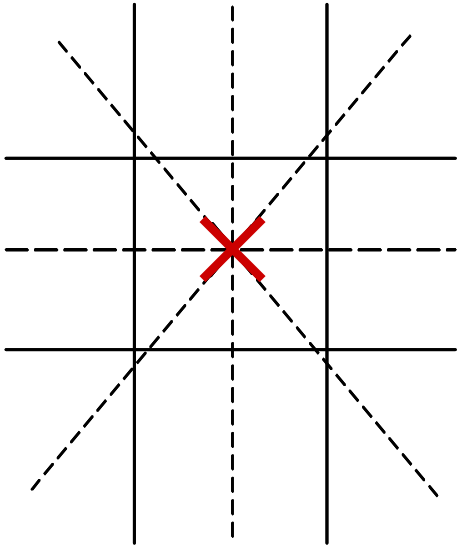
- **启发式搜索实例：一字棋**。在九宫棋盘上，从空棋盘开始，双方轮流在棋盘上摆各自的棋子 × 或 ○（每次一枚），谁先取得三子一线（一行、一列或一条对角线）的结果就取胜。



赢的几率③



赢的几率②



赢的几率④

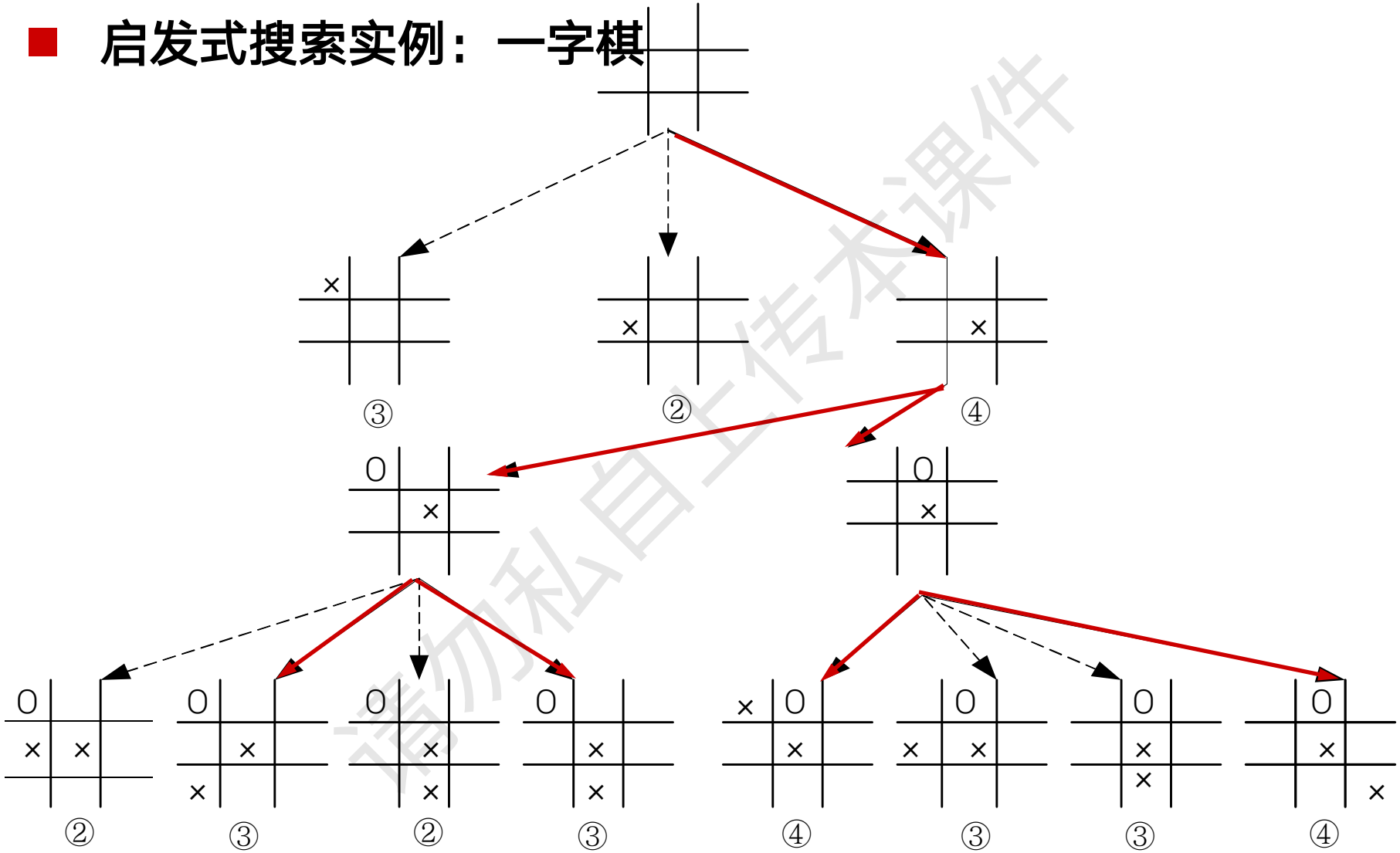
启发式策略的运用

统计该落子点位可能取胜的情况数

请勿私自上传本课件

1.3 启发式搜索

- 启发式搜索实例：一字棋



启发式搜索下缩减的状态空间

请勿私自上传本课件

1.3 启发式搜索

■ 启发信息与估价函数

✓ 估价函数的任务就是**估计待搜索节点的“有希望”程度**，并依次给它们排定次序（在open表中）。

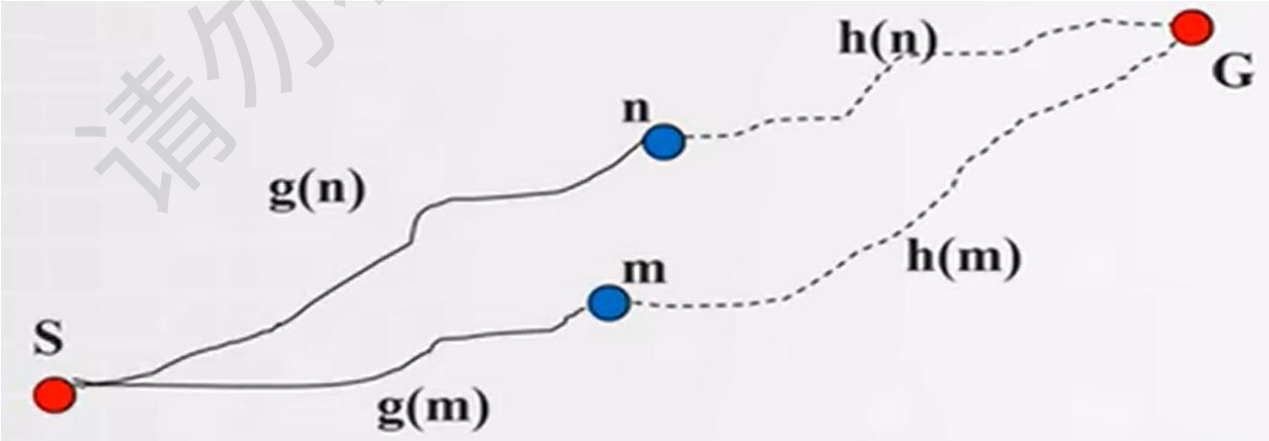
估价函数 $f(n)$ 一般形式是：

路径代价　启发函数

$$f(n) = g(n) + h(n)$$

从起始节点到节点n的实际代价

从节点n到目标的最短路径估计代价



请勿私自传播本课件

1.3 启发式搜索

一般启发式图搜索算法（A算法）

在图搜索算法中，如果能在搜索的每一步都利用估价函数f(n)=g(n)+h(n)对Open表中的节点进行排序，则该搜索算法为A算法。

$$f(n) = g(n) + h(n)$$

启发函数

从起始节点到节点n的实际代价

从节点n到目标的最短路径估计代价

请勿私自上传本课件

1.3 启发式搜索

■ A算法过程描述

(1)把初始节点S_0放入Open表中，$f(S_0)=g(S_0)+h(S_0)$；

(2)如果Open表为空，则问题无解 失败退出

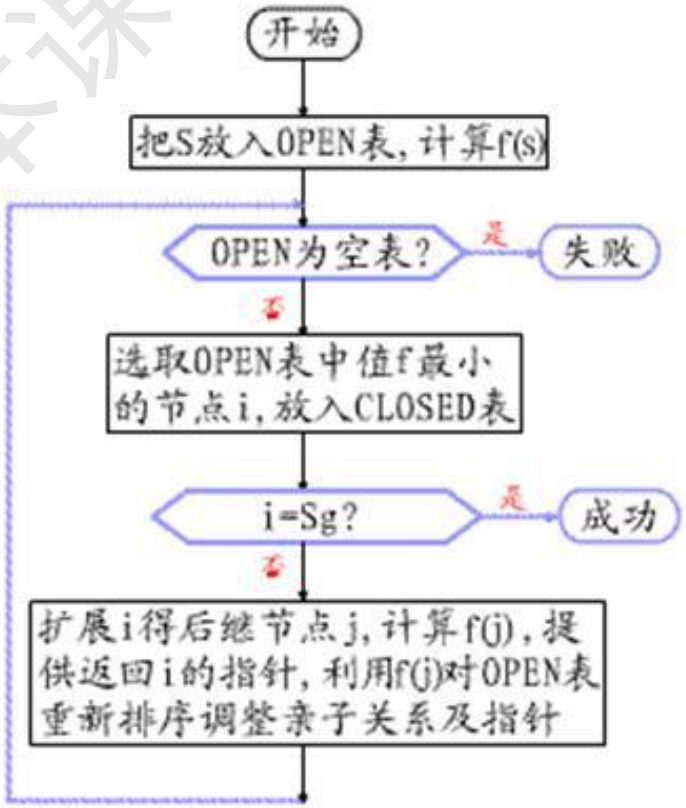
(3)把Open表的第一个节点取出放入Closed表，并记该节点为n；

(4)考察节点n是否为目标节点。若是，则找到了问题的解，成功退出；

(5)扩展节点n，生成其子节点n_i(i=1, 2, …)，**计算每一个子节点的估价值$f(n_i)$(i=1, 2, …)**，并为每一个子节点设置指向父节点的指针，然后将这些子节点放入Open表中；

(6)根据各节点的估价函数值，对Open表中的全部节点按**从小到大的顺序重新进行排序**；

(7)转第(2)步。



禁止自上传本课件

例 八数码问题

设问题的初始状态S_0和目标状态S_g如下图所示，

估价函数为：

f(n)=g(n)+h(n)

g(n)表示节点n在搜索树中的深度

h(n)表示节点n中“不在位”的数码个数。

请用**A算法**解决该问题。

S_0

2	8	3
1	6	4
7		5

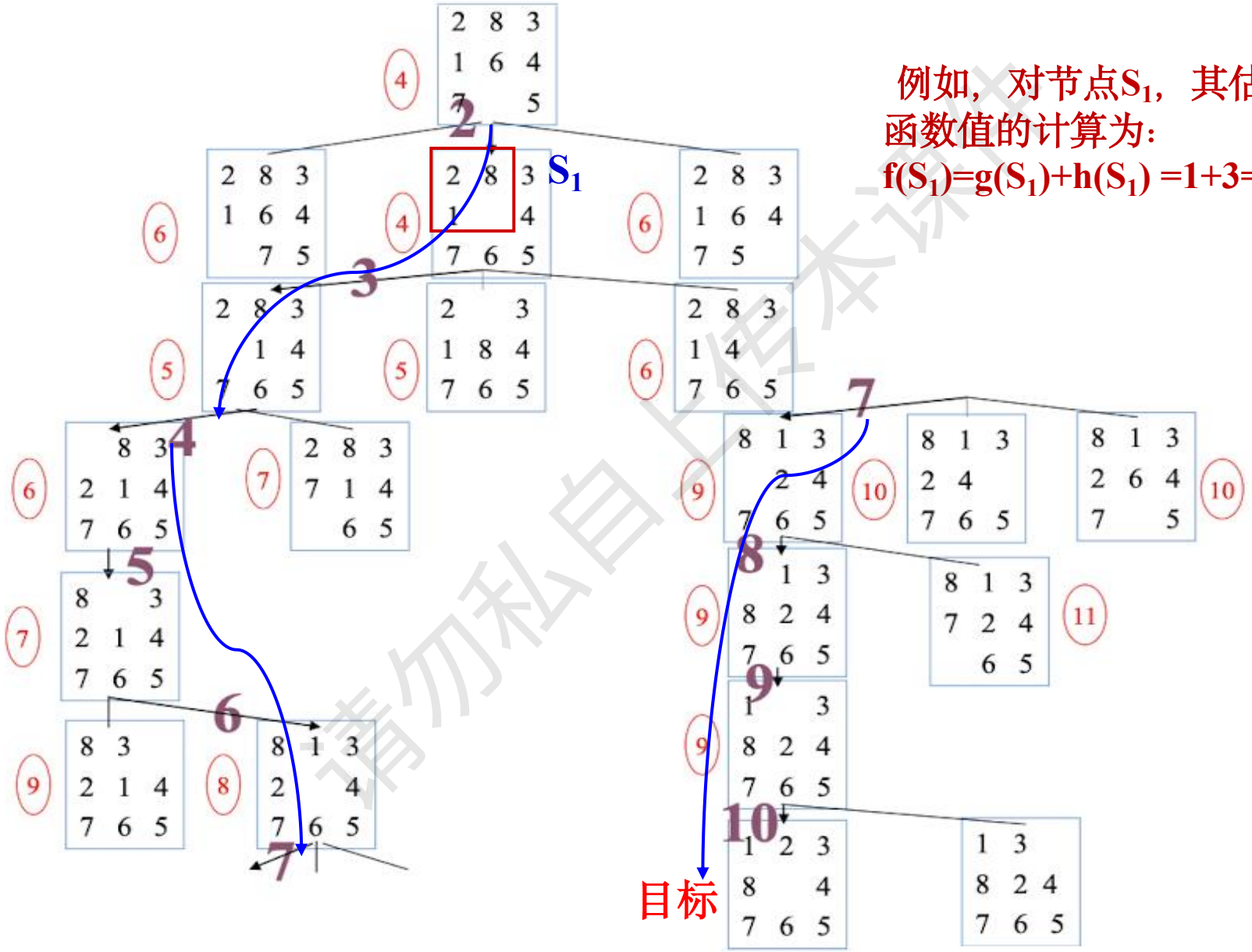
Sg

1	2	3
8		4
7	6	5

请勿私自上传本课件

例 八数码问题(A算法求解)

h(n)：节点n中“不在位”的数码个数



例如，对节点S_1，其估价函数值的计算为：

$f(S_1)=g(S_1)+h(S_1)=1+3=4$

请勿私自上传本课件

1.3 启发式搜索

■ **A*算法**

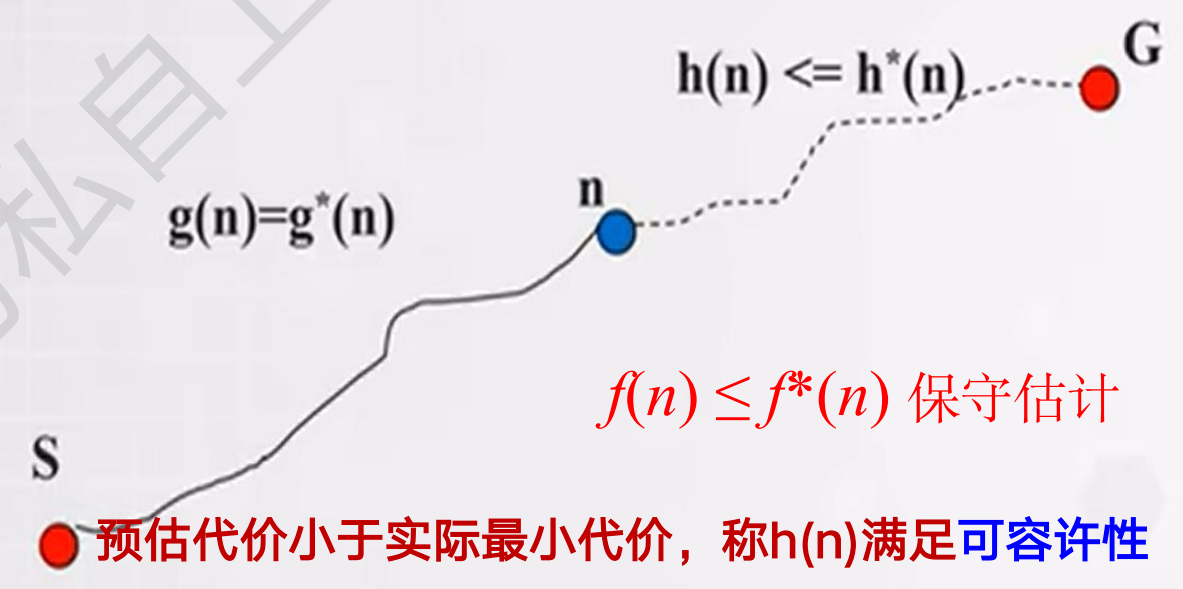
启发函数

$$f(n) = g(n) + h(n)$$

从起始节点到节点n的路径代价　　从节点n到目标的最短路径代价估计值

对A算法的**启发函数**h(n)限制其满足: $h(n) \leqslant h^*(n)$

h*(n)是从节点n到目标节点的**实际最短路径代价**；h(n)的值不大于节点n到目标节点实际最短路径代价值,即h(n)**不会高估**到达目标节点的代价!



$f(n) \leq f^*(n)$ 保守估计

预估代价小于实际最小代价，称h(n)满足可容许性

请勿私自上传本课件

1.3 启发式搜索

■ A*算法

可容许的h(n)启发式函数，A*一定是代价最优的

反证法：假设最优路径代价为C*，但是该算法返回的路径代价为C>C*，那么最优路径上一定存在某个未扩展的节点n（如果最优路径上的所有节点都被扩展，那么算法返回的就是C*，而不会大于它）。

f(n)>C* （否则n将是已经扩展节点）

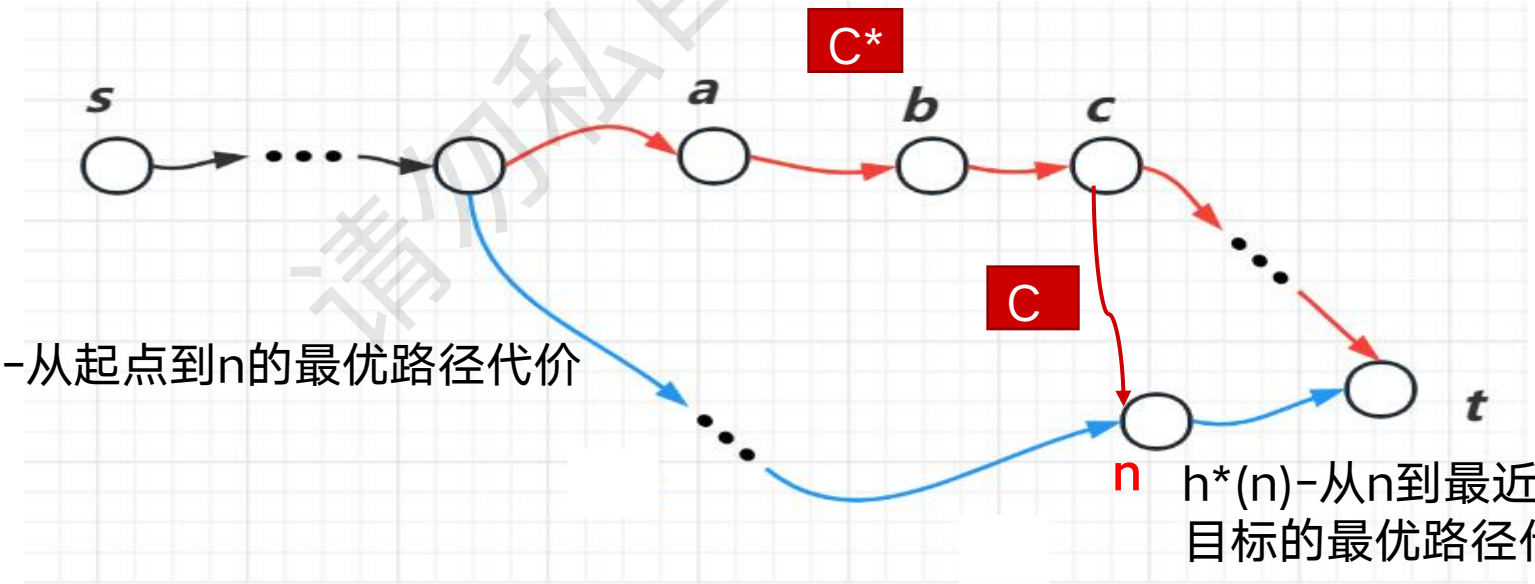
f(n)=g(n)+h(n) (根据定义)

f(n)=g*(n)+h(n) (因为n在一条最优路径上)

f(n)<=g*(n)+h*(n) (根据可容许性质，h(n)<=h*(n))

f(n)<=C* (根据定义 C*=g*(n)+h*(n))

得出矛盾结论，反证了A*一定只返回代价最优路径！



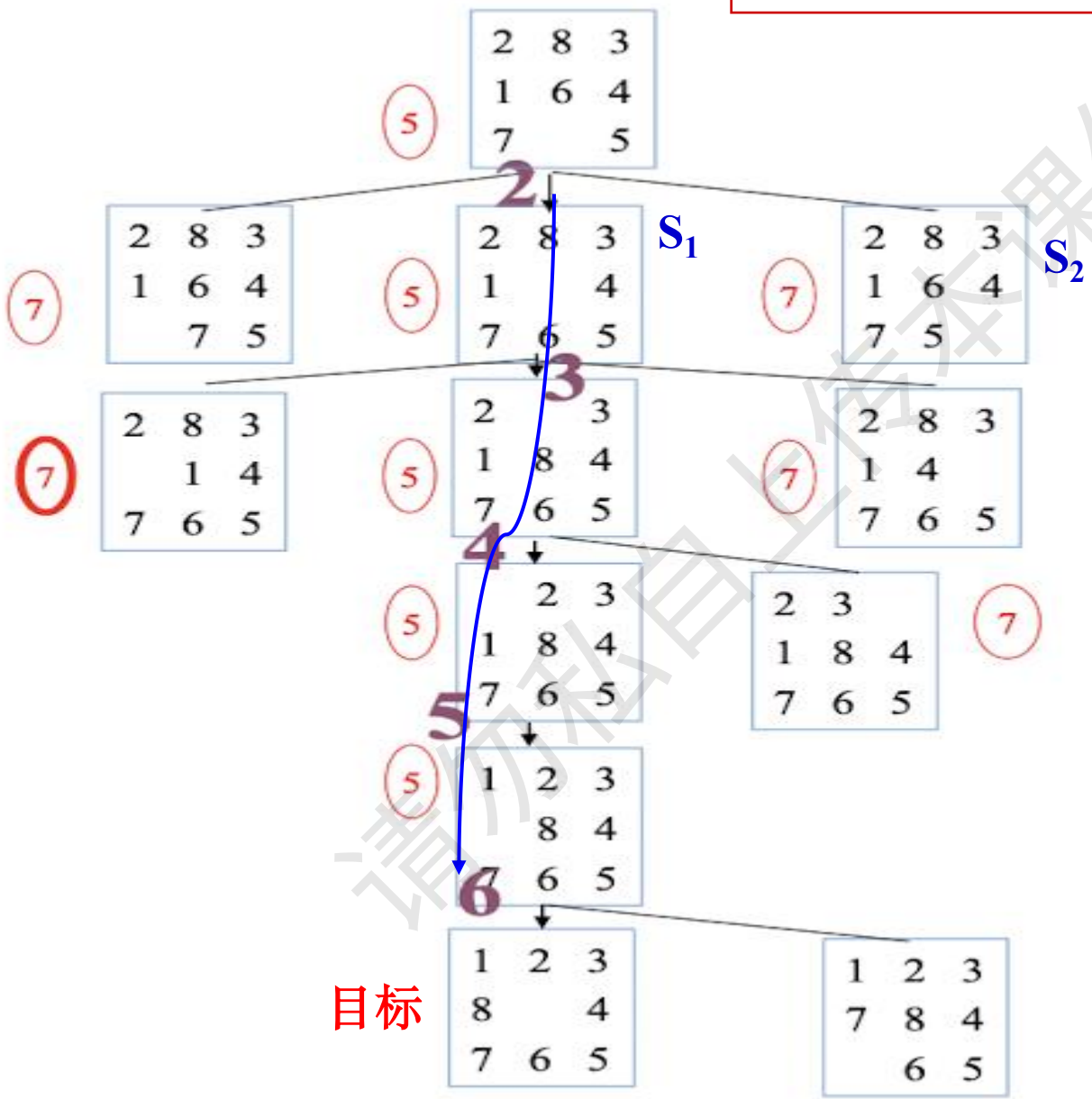
g*(n)-从起点到n的最优路径代价

h*(n)-从n到最近目标的最优路径代价

请勿私自上传本课件

例 八数码问题(A*求解)

h(n)：节点n中“不在位”的距离和
相比“不在位”的数码个数，更接近h*



例如，对节点S_1，其估价函数值的计算为：
$f(S_1)=g(S_1)+h(S_1)=1+4=5$

例如，对节点S_2，其估价函数值的计算为：
$f(S_2)=g(S_2)+h(S_2)=1+6=7$

请勿私自上传本课件

1.3 启发式搜索

A*算法流程与伪码描述

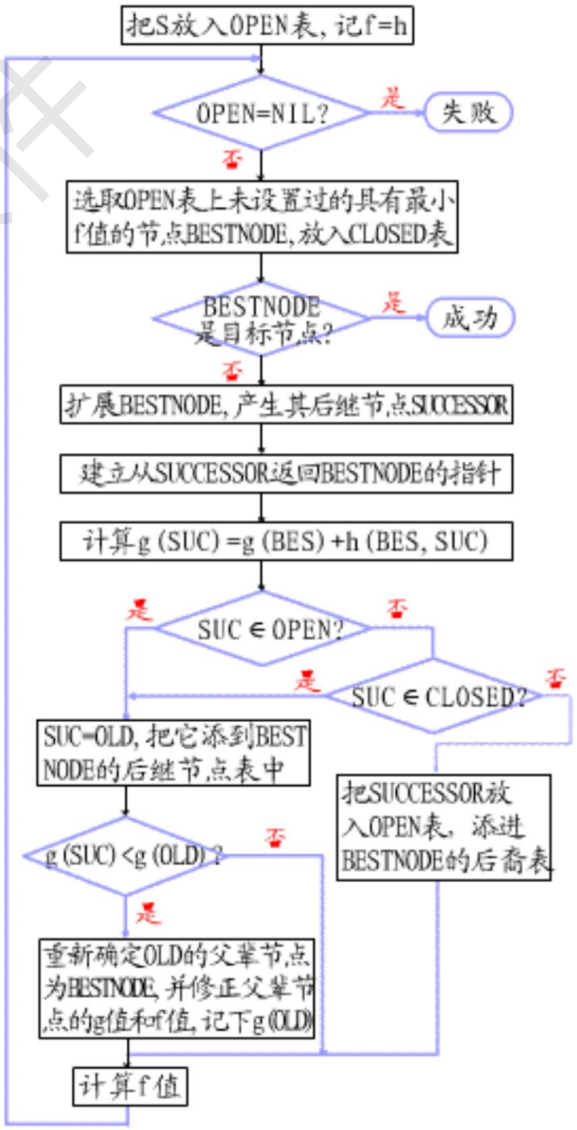
```
Put the start node, s, on a list called OPEN of unexpanded nodes
if |OPEN| = 0 then
    Exit—no solution exists
Remove a node n from OPEN, at which f = g + h is minimum and place it
on a list called CLOSED
if n is a goal node then
    Exit with solution
Expand node n, generating all its successors with pointers back to n
for all successor n' of n do
    Calculate f(n')
    if n' ∉ OPEN AND n' ∉ CLOSED then
        Add n' to OPEN
        Assign the newly computed f(n') to node n'
    else
        If new f(n') value is smaller than the previous value, then update with
        the new value (and predecessor)
        If n' was in CLOSED, move it back to OPEN
Go to (2)
```

请勿私自上传本课件

1.3 启发式搜索

A*算法流程与伪码描述

1: Put the start node, s, on a list called $OPEN$ of unexpanded nodes
2: **if** $|OPEN| = 0$ **then**
3: Exit—no solution exists
4: Remove a node n from $OPEN$, at which $f = g + h$ is minimum and place it on a list called $CLOSED$
5: **if** n is a goal node **then**
6: Exit with solution
7: Expand node n, generating all its successors with pointers back to n
8: **for all** successor n' of n **do**
9: Calculate $f(n')$
10: **if** $n' \notin OPEN$ AND $n' \notin CLOSED$ **then**
11: Add n' to $OPEN$
12: Assign the newly computed $f(n')$ to node n'
13: **else**
14: If new $f(n')$ value is smaller than the previous value, then update with the new value (and predecessor)
15: If n' was in $CLOSED$, move it back to $OPEN$
16: Go to (2)



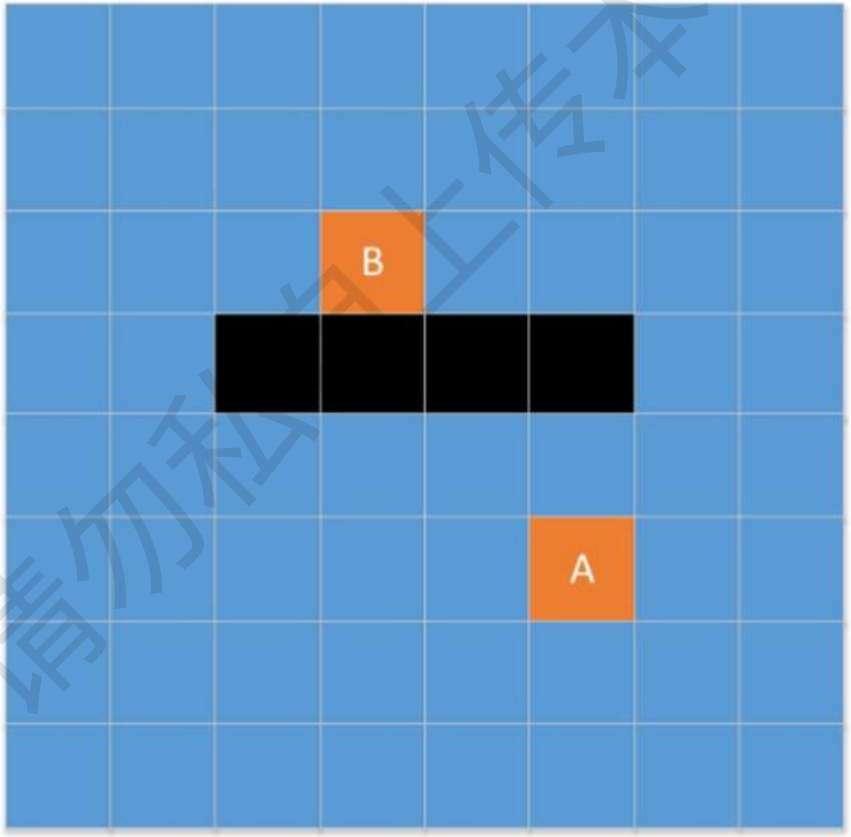
请勿私自上传本课件

课堂练习

A*搜索实现机器人路径规划

A为起点，B为终点，黑色块为障碍物

写出分析设计过程，给出最佳路径



请勿私自上传本课件

课堂练习

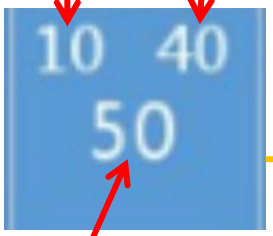
移动距离：

10：直线走一格

14：斜线走一格

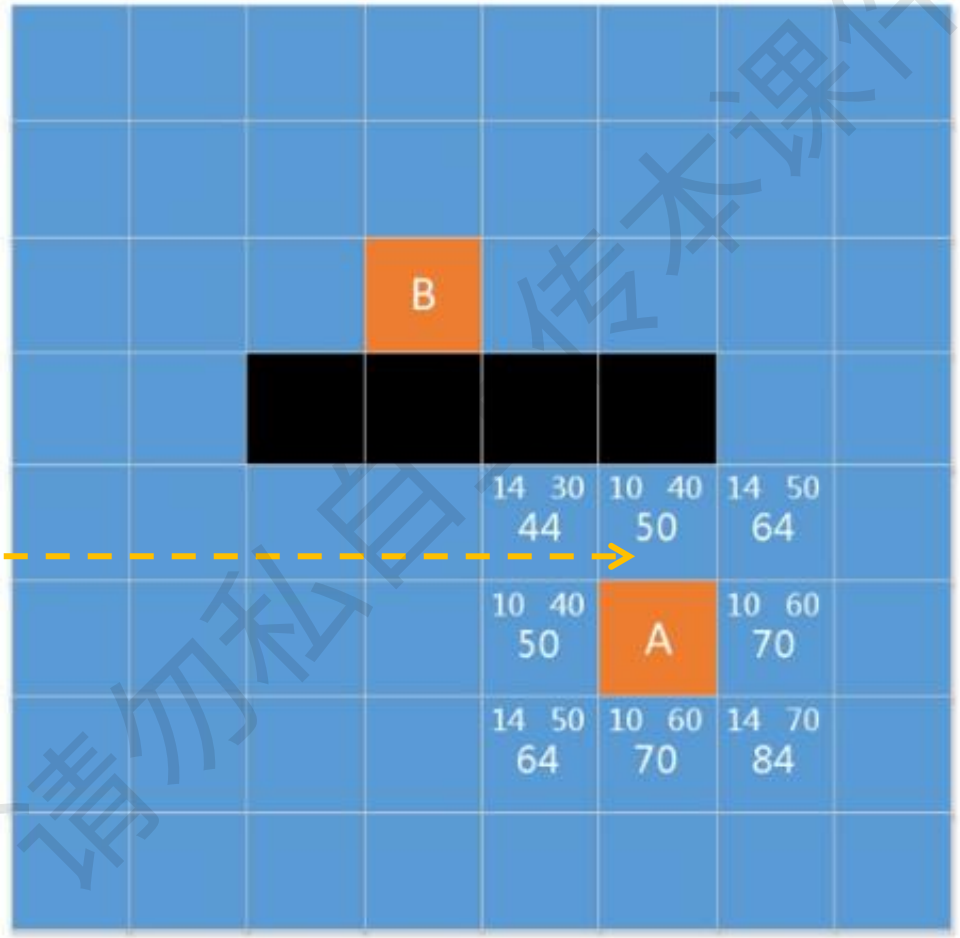
g(n)路径代价

h(n)启发函数



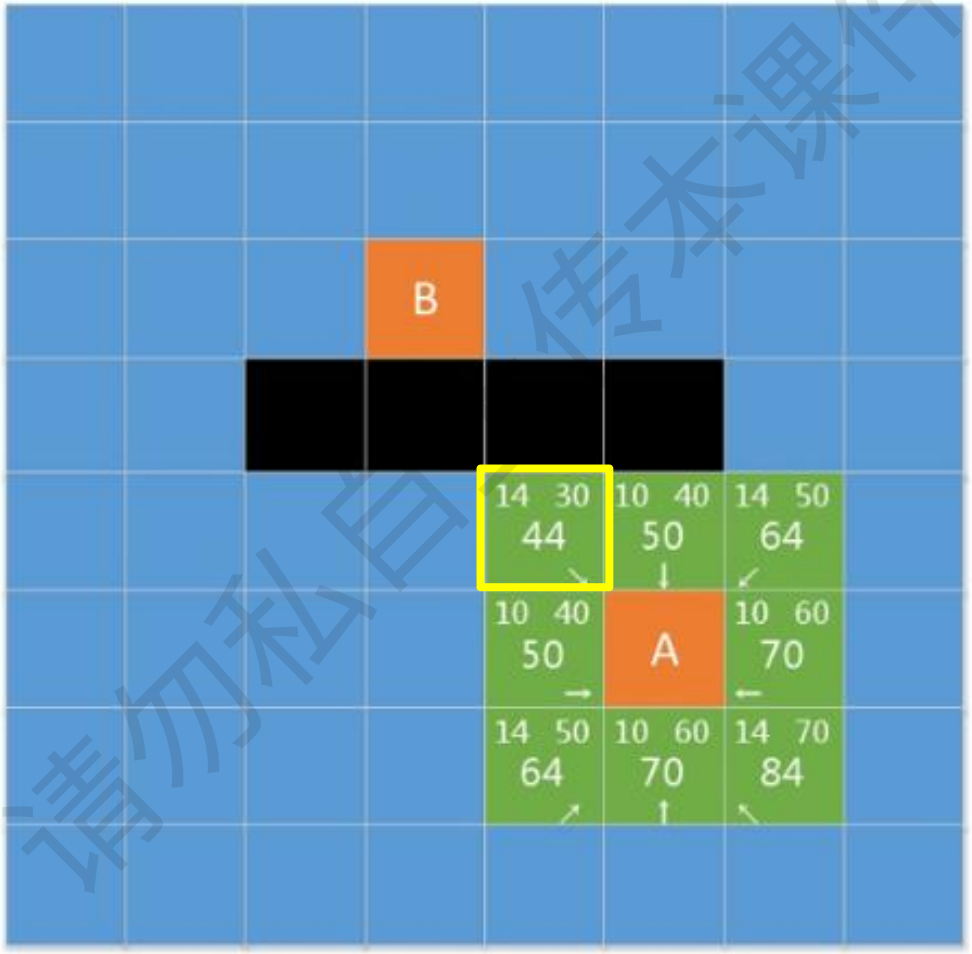
f(n)=g(n)+h(n)

估价函数



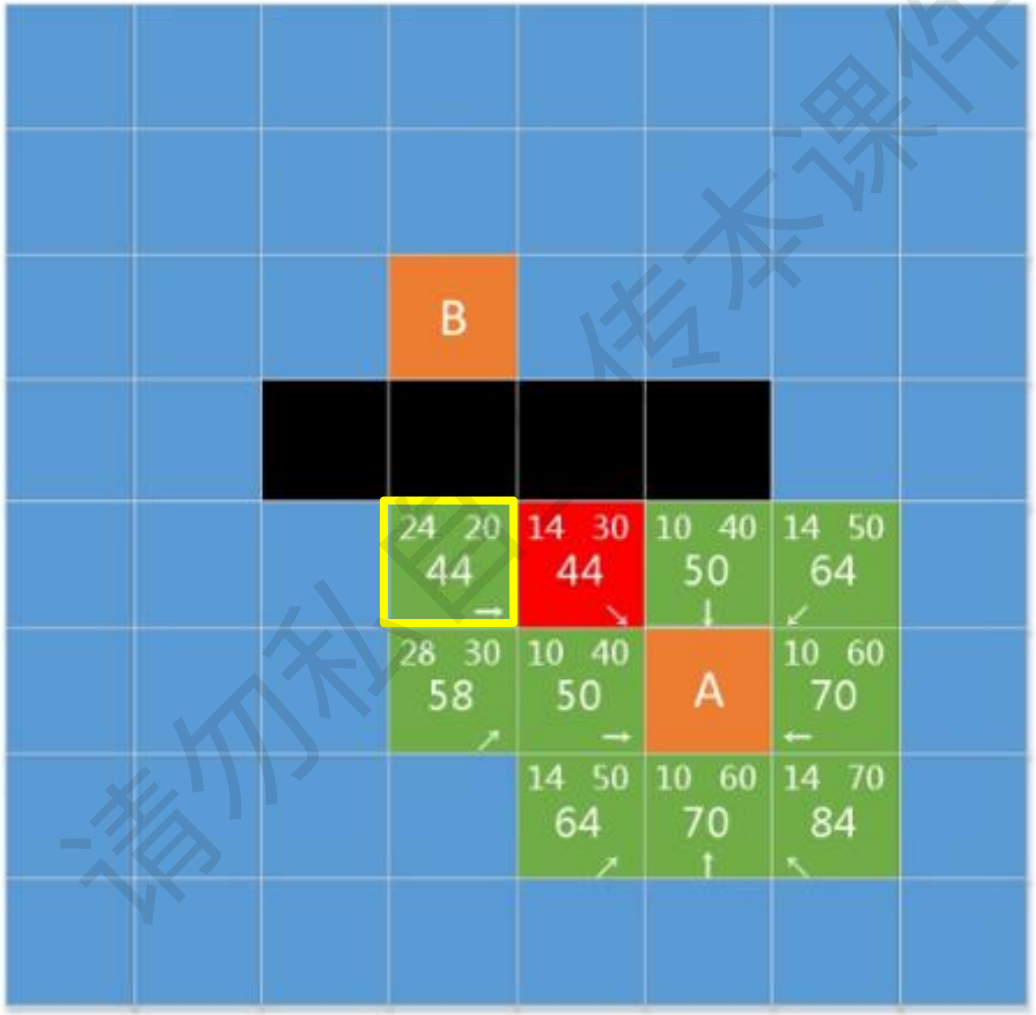
请勿私自传播课件

课堂练习



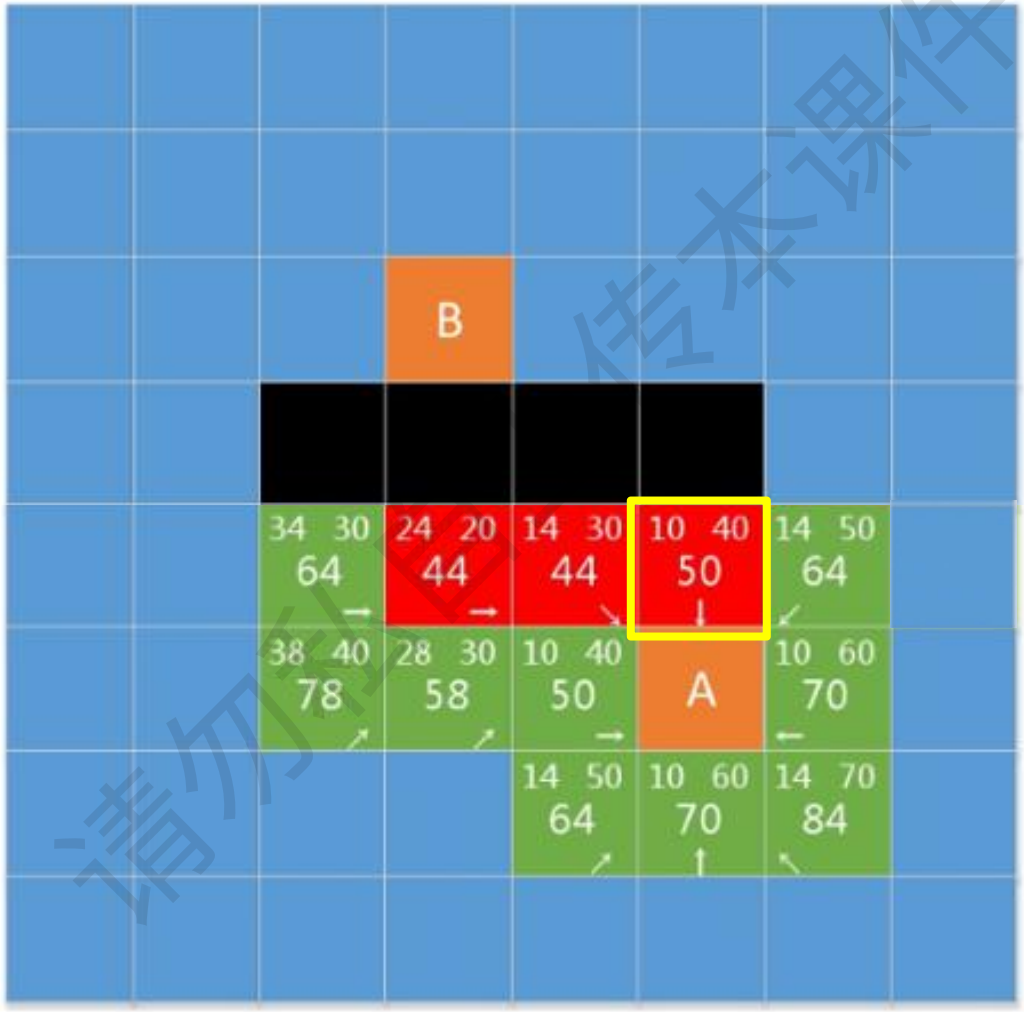
请勿私自传播课件

课堂练习



请勿私自传本课件

课堂练习

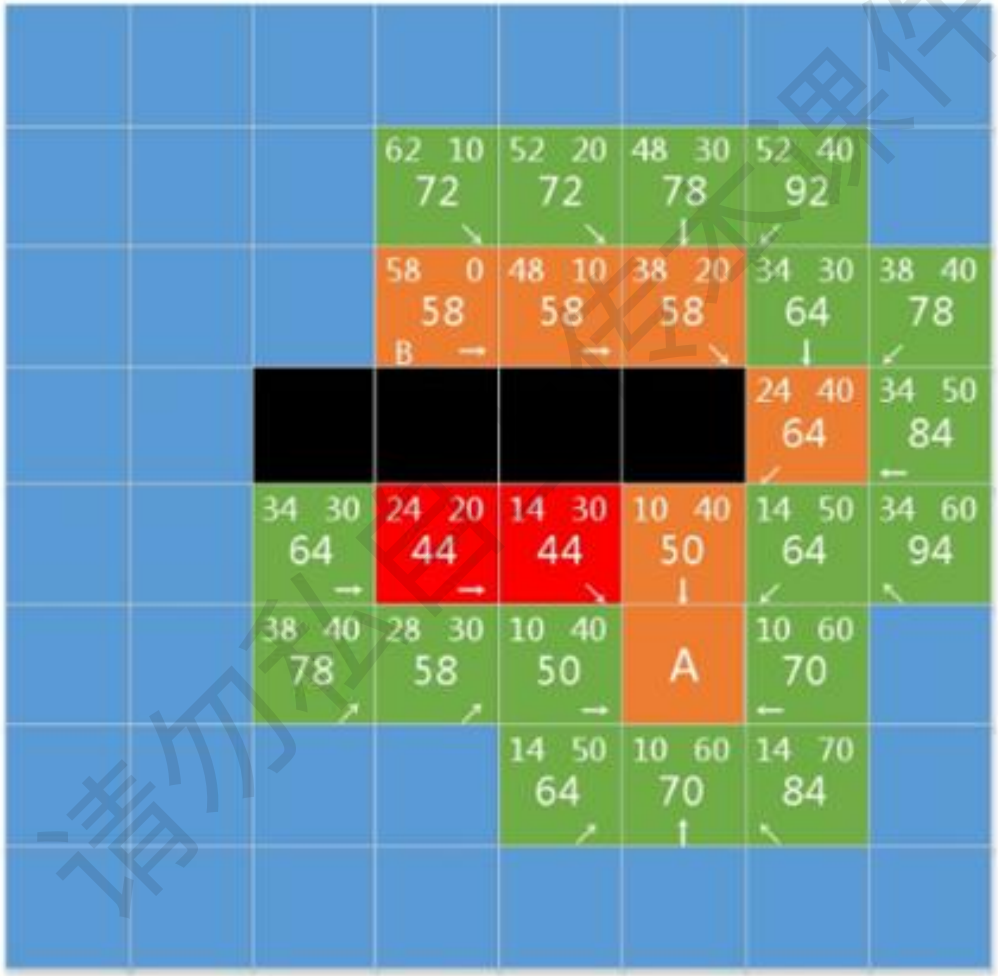


请勿转载 传本课件

课堂练习

请勿转载 本系课件

课堂练习



请勿私自传播课件

搜索求解策略

1.1 搜索的概念

1.2 盲目搜索

1.3 启发式搜索

1.4 博弈树搜索

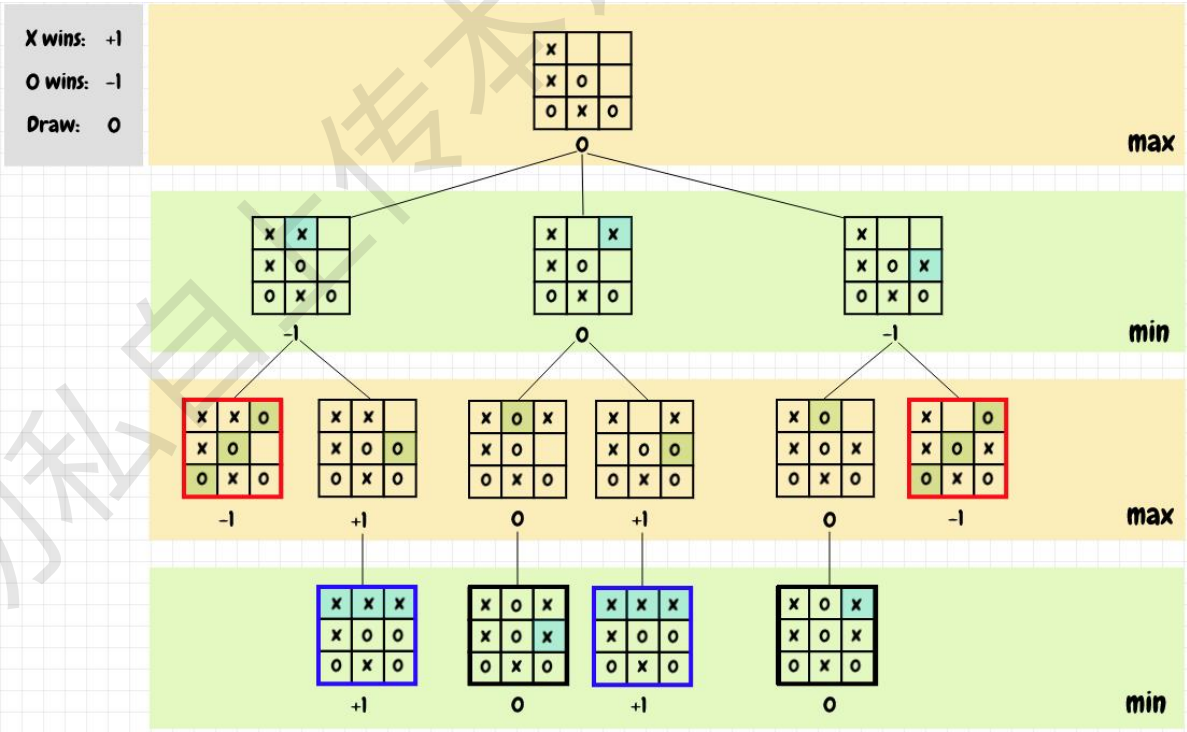
请勿私自上传本课件

1.4 博弈树搜索

■ 博弈概述

诸如下棋、打牌、竞技、战争等一类竞争性活动称为博弈，如最简单的零和博弈，具有如下特征：

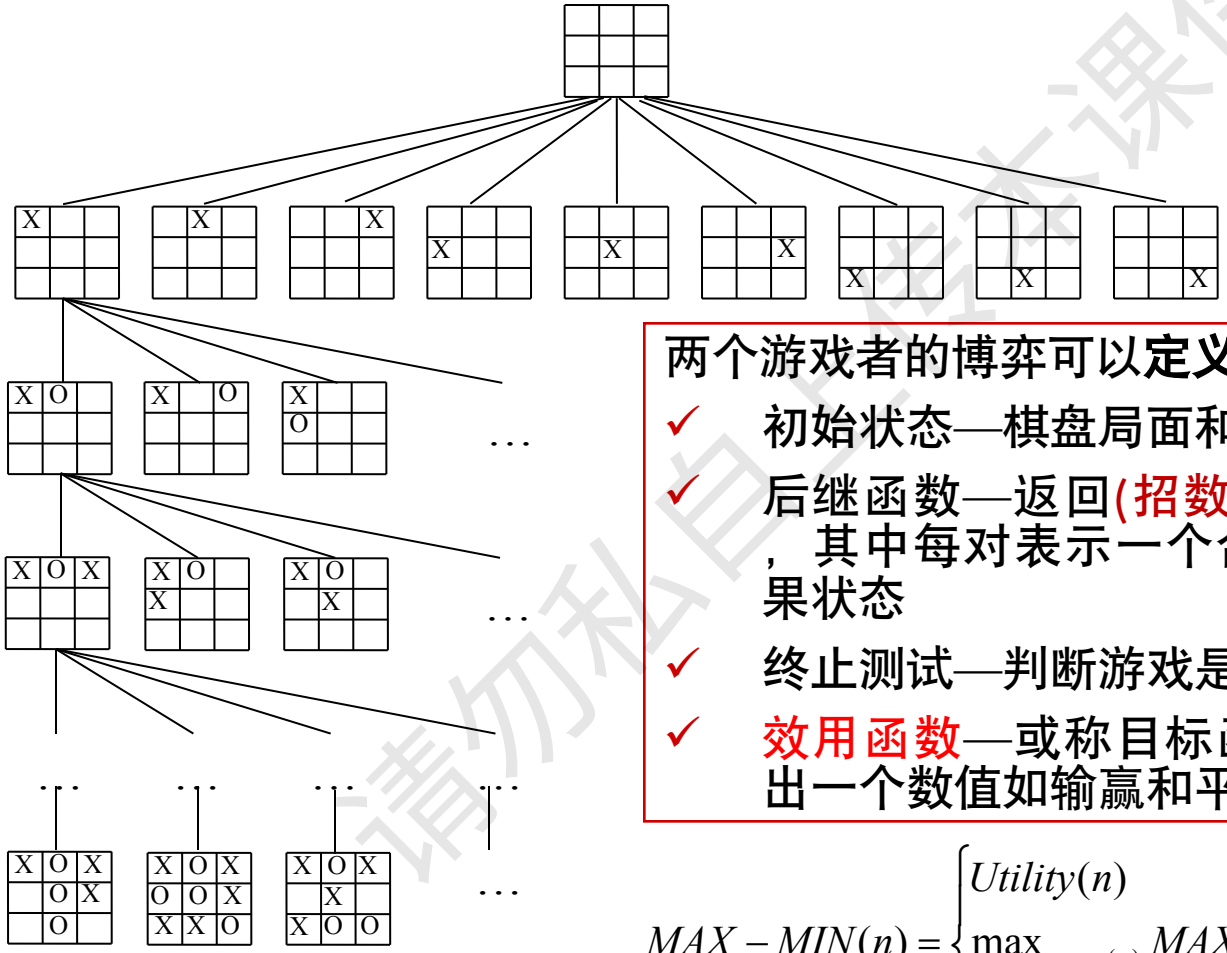
- 对垒的MAX、MIN双方轮流采取行动，博弈的结果只有三种情况：MAX方胜，MIN方败；MIN方胜，MAX方败；和局
- 在对垒过程中，任何一方都了解当前的格局及过去的历史；任何一方在采取行动前都要根据当前的实际情况，选取对自己为最有利而对对方最为不利的对策。



仅供私自上传本课件

1.4 博弈树搜索

■ **博弈搜索** 以一字棋为例-搜索问题定义



-1　　0　　+1

效用

两个游戏者的博弈可以**定义为如下搜索问题**：

- ✓ 初始状态—棋盘局面和哪个游戏者出招
- ✓ 后继函数—返回(招数,状态)对的一个列表，其中每对表示一个合法招数和相应的结果状态
- ✓ 终止测试—判断游戏是否结束
- ✓ 效用函数—或称目标函数，对终止状态给出一个数值如输赢和平局(以-1/+1/0表示)

$$MAX-MIN(n)=\begin{cases} Utility(n) & \text{当n为终止状态} \\ \max_{s\in son(n)} MAX-MIN(s) & \text{当n为MAX节点} \\ \min_{s\in son(n)} MAX-MIN(s) & \text{当n为MIN节点} \end{cases}$$

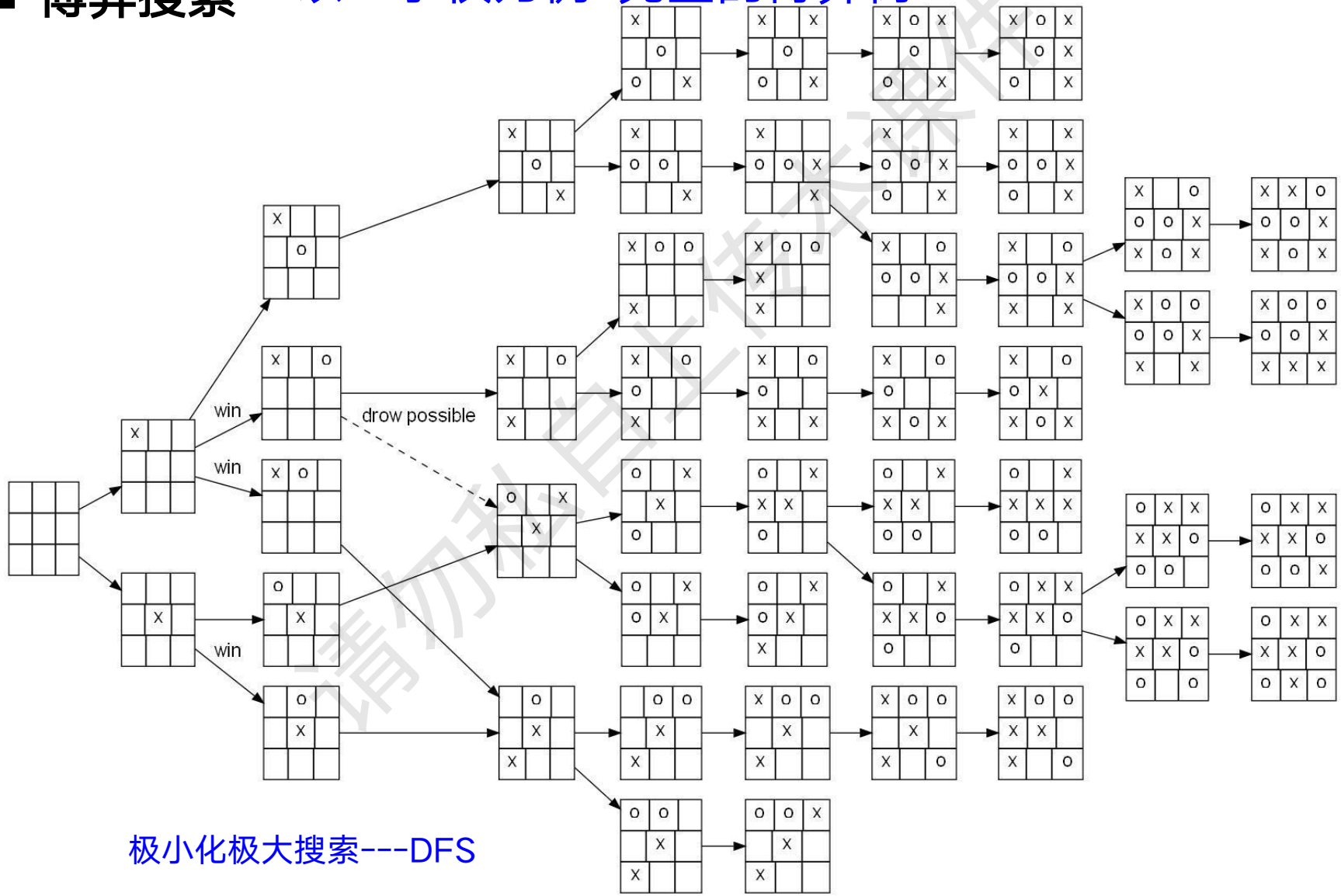
极小化极大搜索算法

请勿私自上传本课件

1.4 博弈树搜索

■ 博弈搜索

以一字棋为例-完整的博弈树

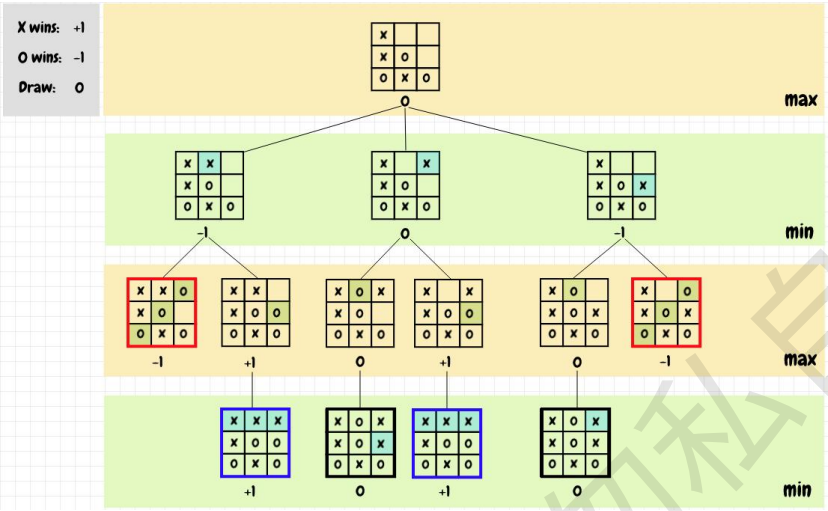


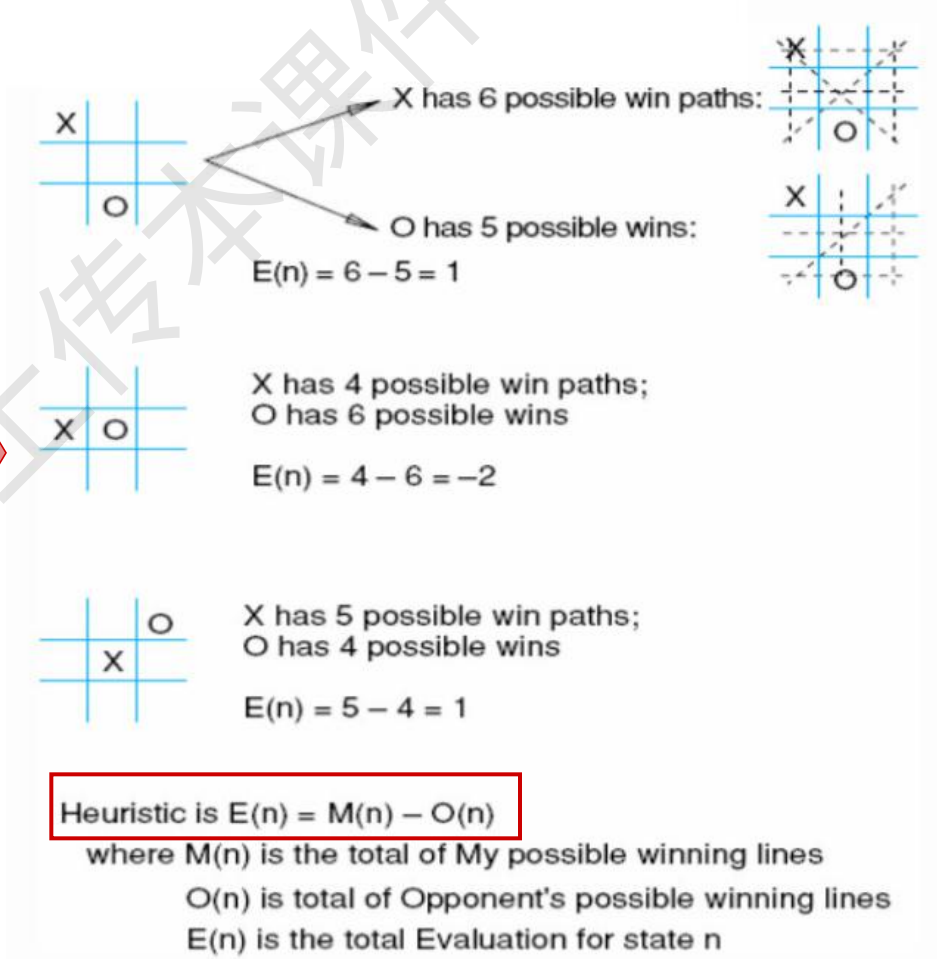
极小化极大搜索---DFS

请勿转自上传课件

1.4 博弈树搜索

■ 博弈搜索　一字棋实例





搜索过程中的启发信息定义？

对于终局情况：

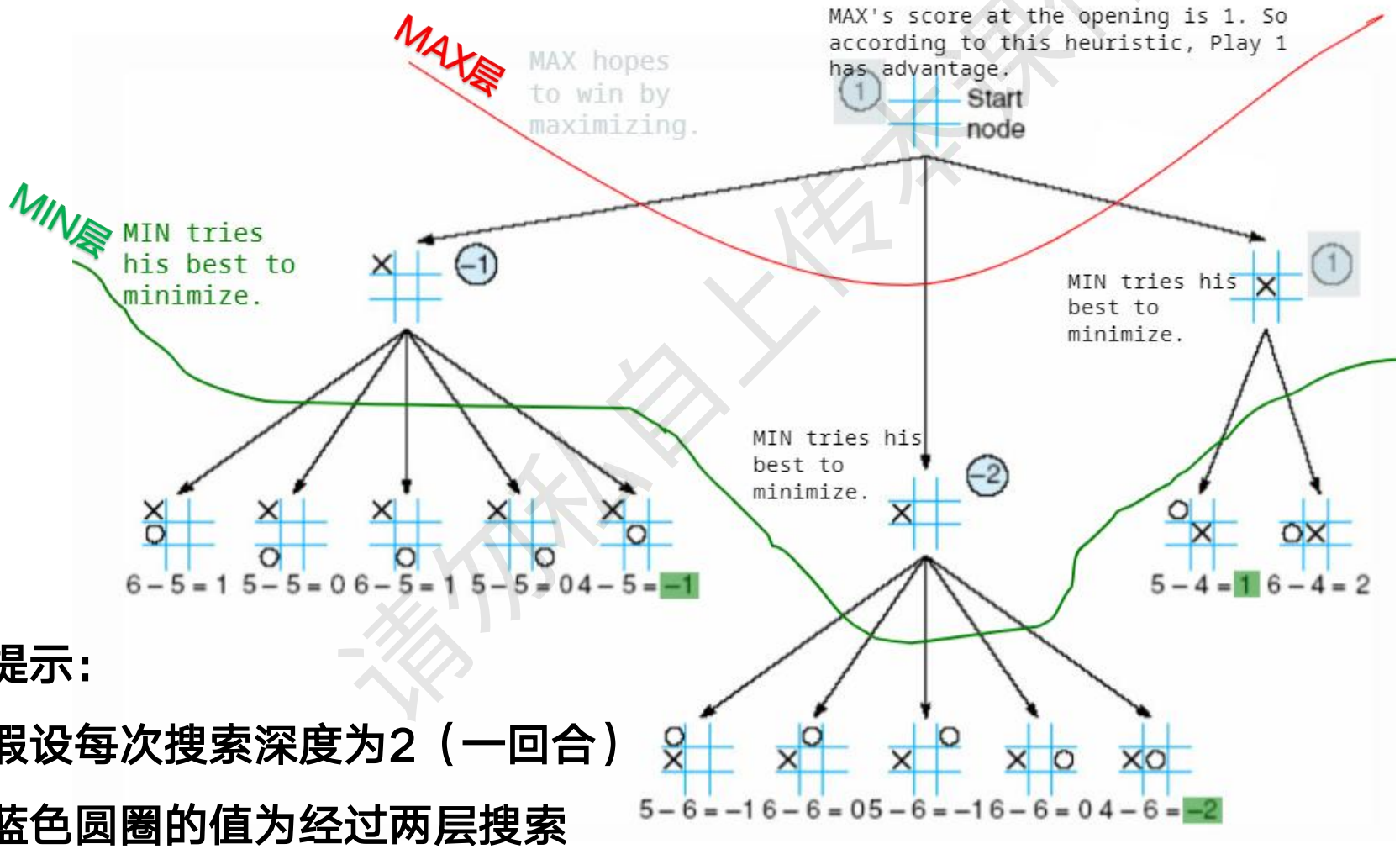
X方赢，启发信息为+∞

O方赢，启发信息为-∞

请勿私自上传本课件

1.4 博弈树搜索

■ 博弈搜索　一字棋实例



请勿私自上传本课件

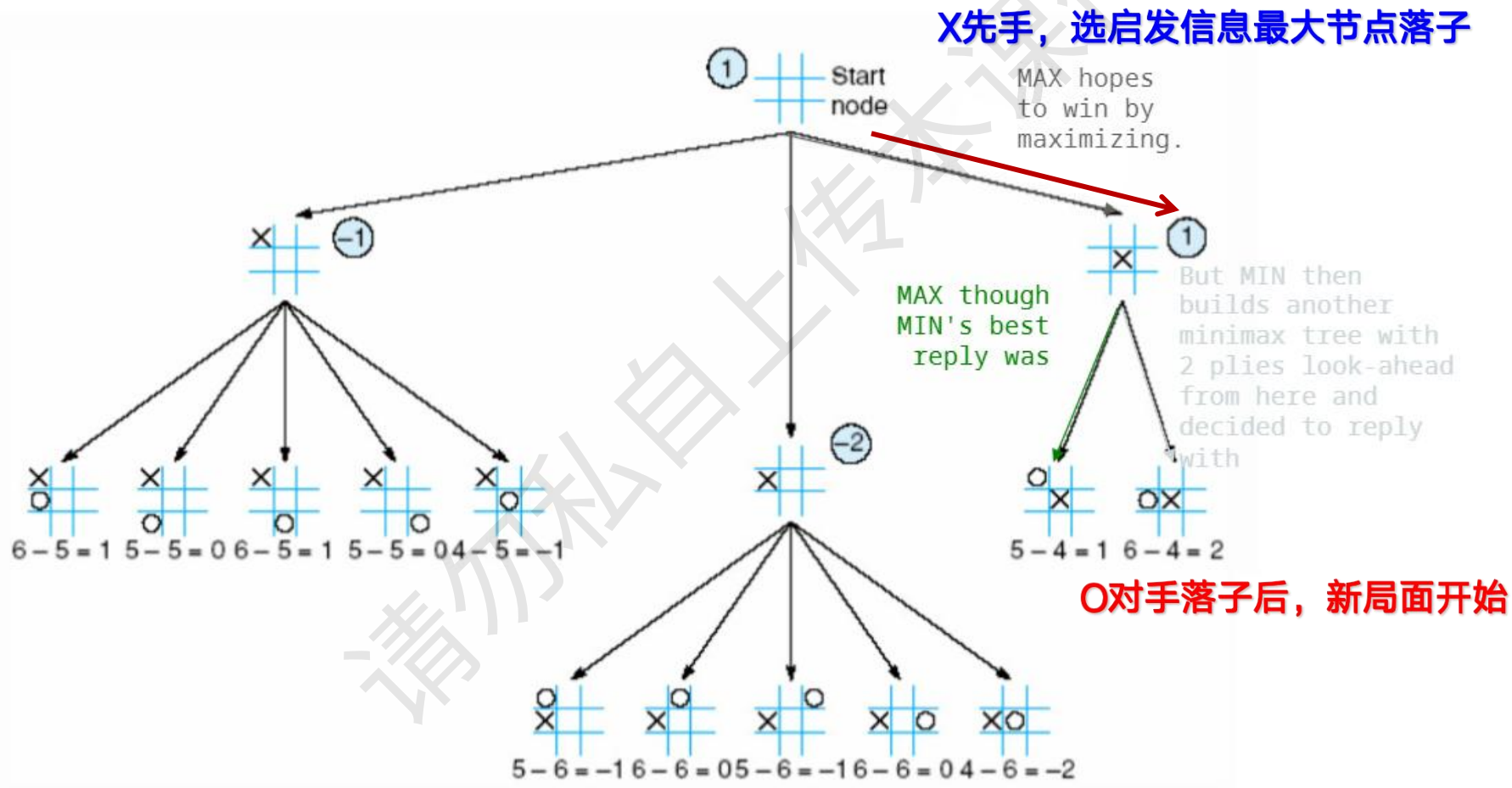
提示：

假设每次搜索深度为2（一回合）

蓝色圆圈的值为经过两层搜索后得到的启发信息

1.4 博弈树搜索

■ 博弈搜索　　一字棋实例

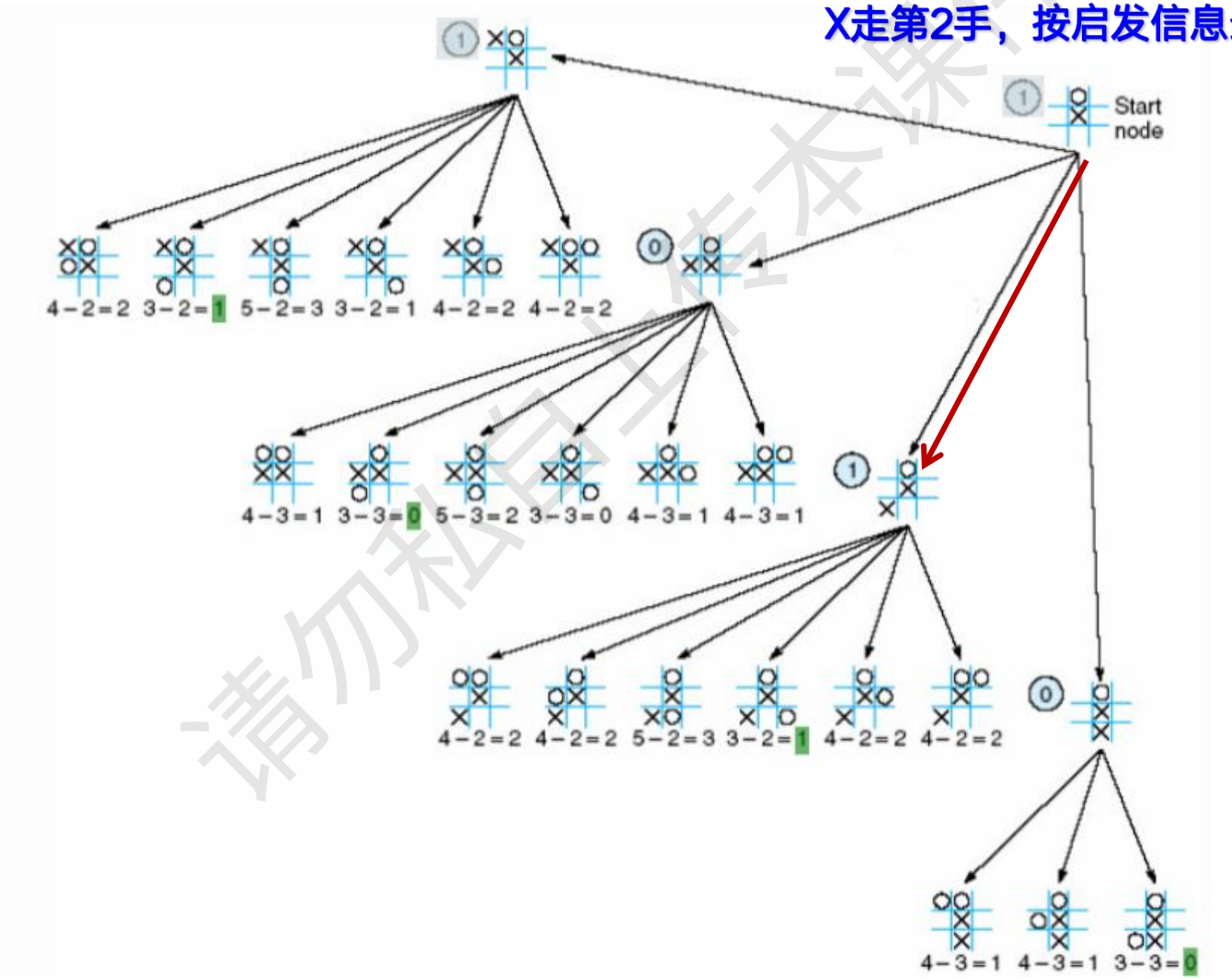


请勿私自上传本课件

1.4 博弈树搜索

■ 博弈搜索　　一字棋实例

X走第2手，按启发信息最大路径落子

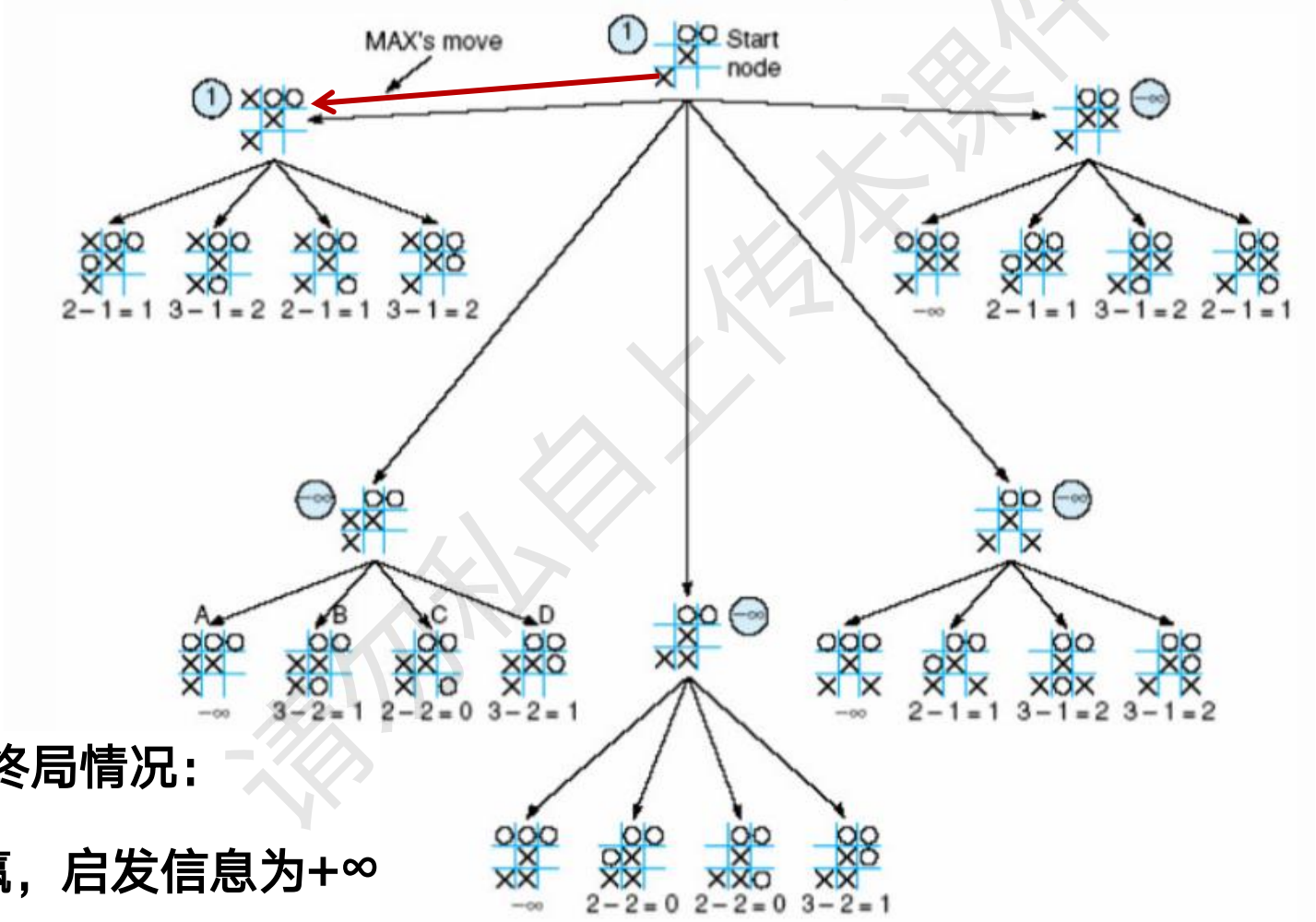


请勿私自上传本课件

1.4 博弈树搜索

■ 博弈搜索　一字棋实例

X走第3手（接近终局），按启发信息最大路径落子



对于终局情况：

X方赢，启发信息为+∞

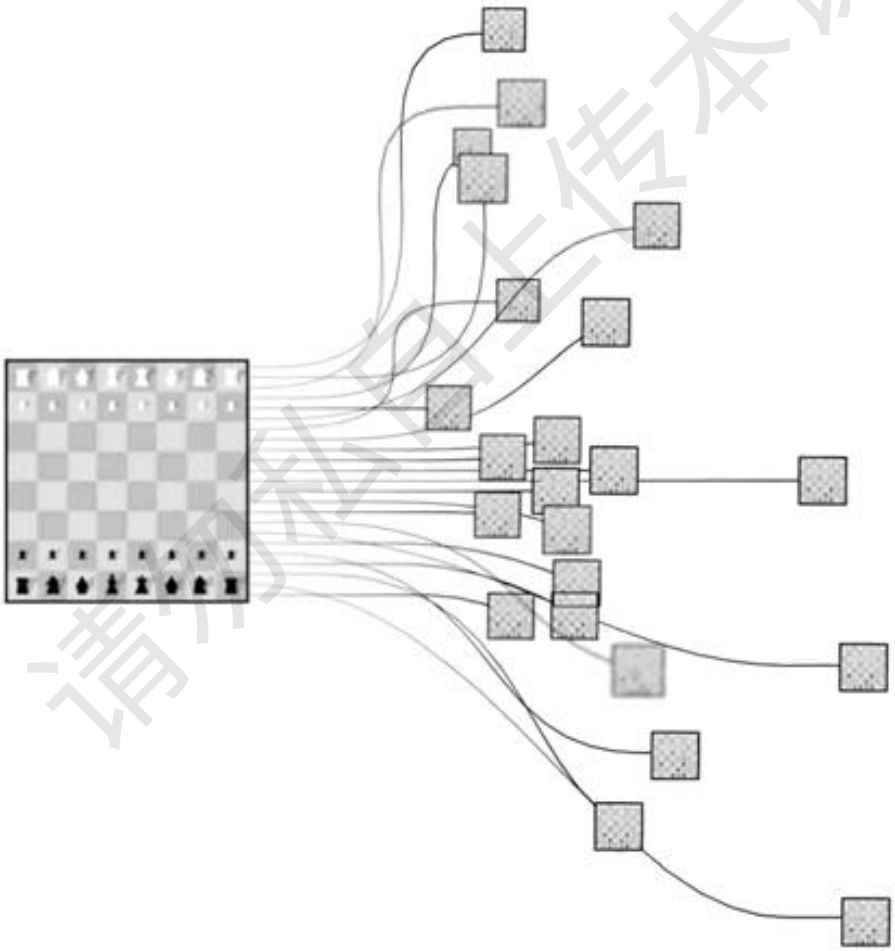
O方赢，启发信息为-∞

请勿私自上传本课件

1.4 博弈树搜索

■ 博弈搜索　　α-β剪枝算法设计思想

博弈状态关于树的深度是指数级的，难有算法可以消除指数项

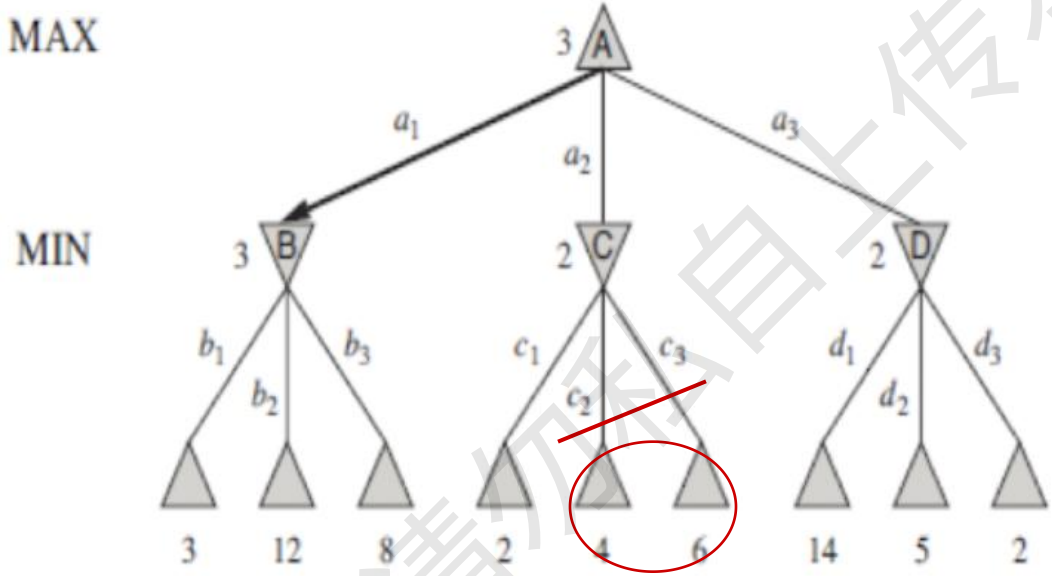


请勿私自传播课件

1.4 博弈树搜索

■ 博弈搜索　α-β剪枝算法设计思想

博弈状态关于树的深度是指数级的，难有算法可以消除指数项
但可通过**剪枝**消除对结果没有影响的树的大部分分支，该策略称为**α-β剪枝**



$$MINIMAX(root)$$
$$= \max(\min(3,12,8), \min(2,x,y), \min(14,5,2))$$
$$= \max(3, \min(2,x,y), 2)$$
$$= \max(3,z,2)=3$$

$$where\ z = \min(2,x,y) \le 2$$

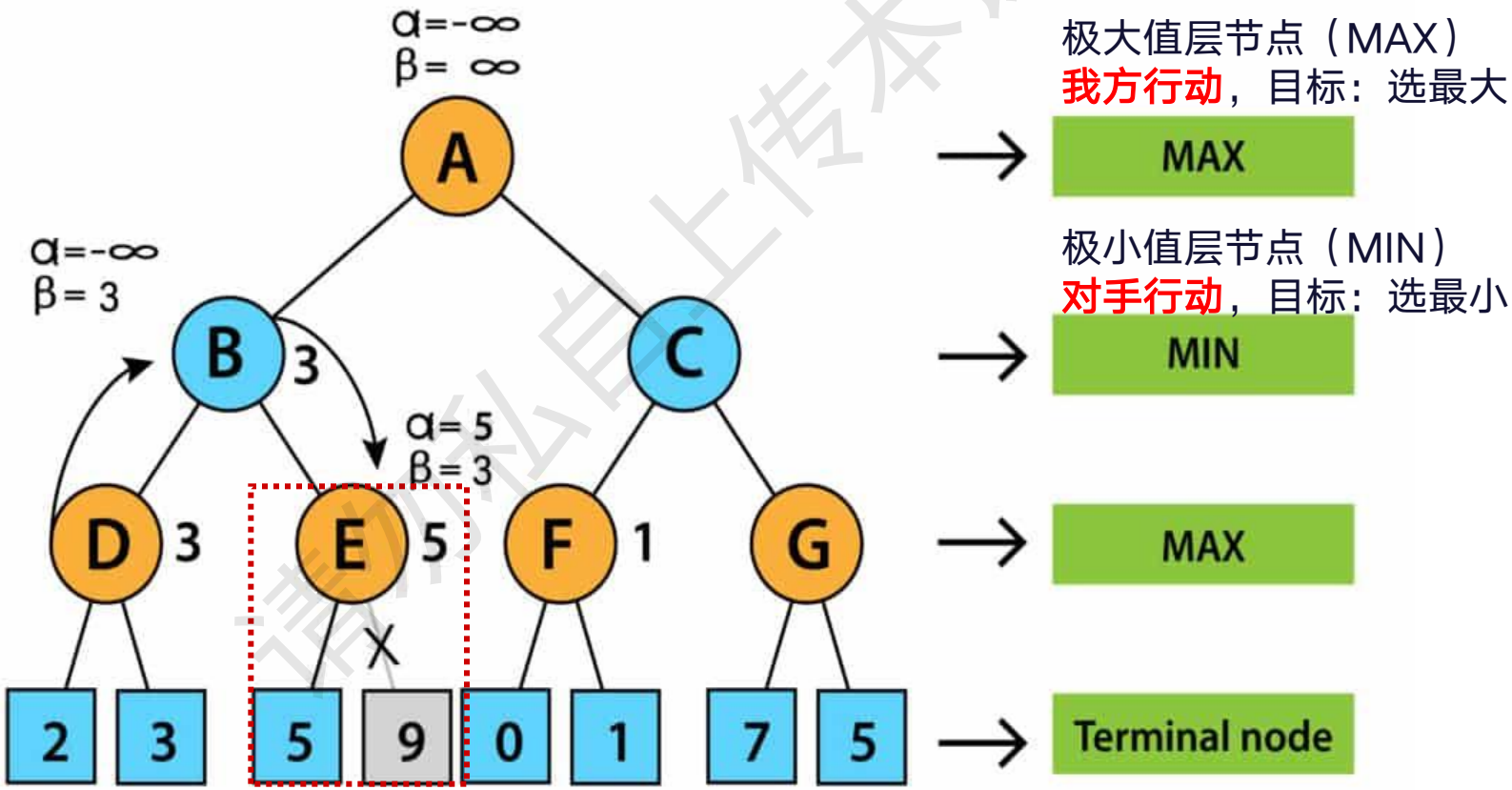
根节点的值以及极小化极大决策与叶节点x和y的值无关！

请勿自上传本课件

1.4 博弈树搜索

■ 博弈搜索　　α-β剪枝算法设计思想（基于启发信息的剪枝）

➢ β剪枝：若任一极大值层节点的α值大于或等于它任一先辈极小值层节点的β值，及α(后继层)≥β(先辈层)，则可中止该极大值层中这个MAX节点以下的搜索过程。这个MAX节点最终的倒推值就设为这个α值。



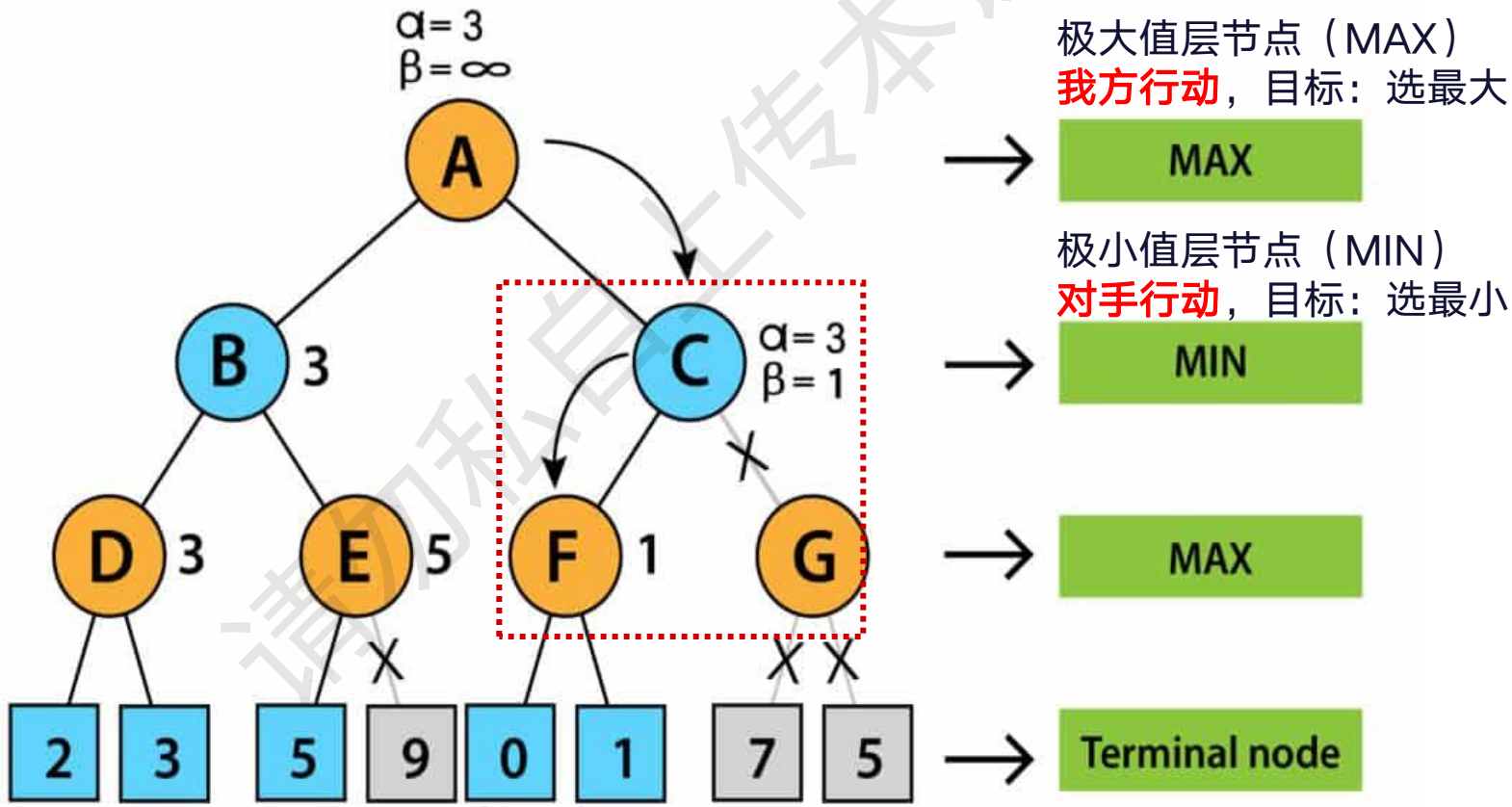
α=到目前为止，在 MAX 层找到的最佳（最大）选择值，即“我至少能获得多少”。为极大层的下界

β=到目前为止，在 MIN 层找到的最差（最小）选择值，即“对手最多让我得到多少”。为极小层的上界

1.4 博弈树搜索

■ 博弈搜索　　α-β剪枝算法设计思想（基于启发信息的剪枝）

➢ α剪枝：若任一极小值层节点的β值小于或等于它任一先辈极大值层节点的α值，及α(先辈层)≥β(后继层)，则可中止该极小值层中这个MIN节点以下的搜索过程。这个MIN节点最终的倒推值就设为这个β值。



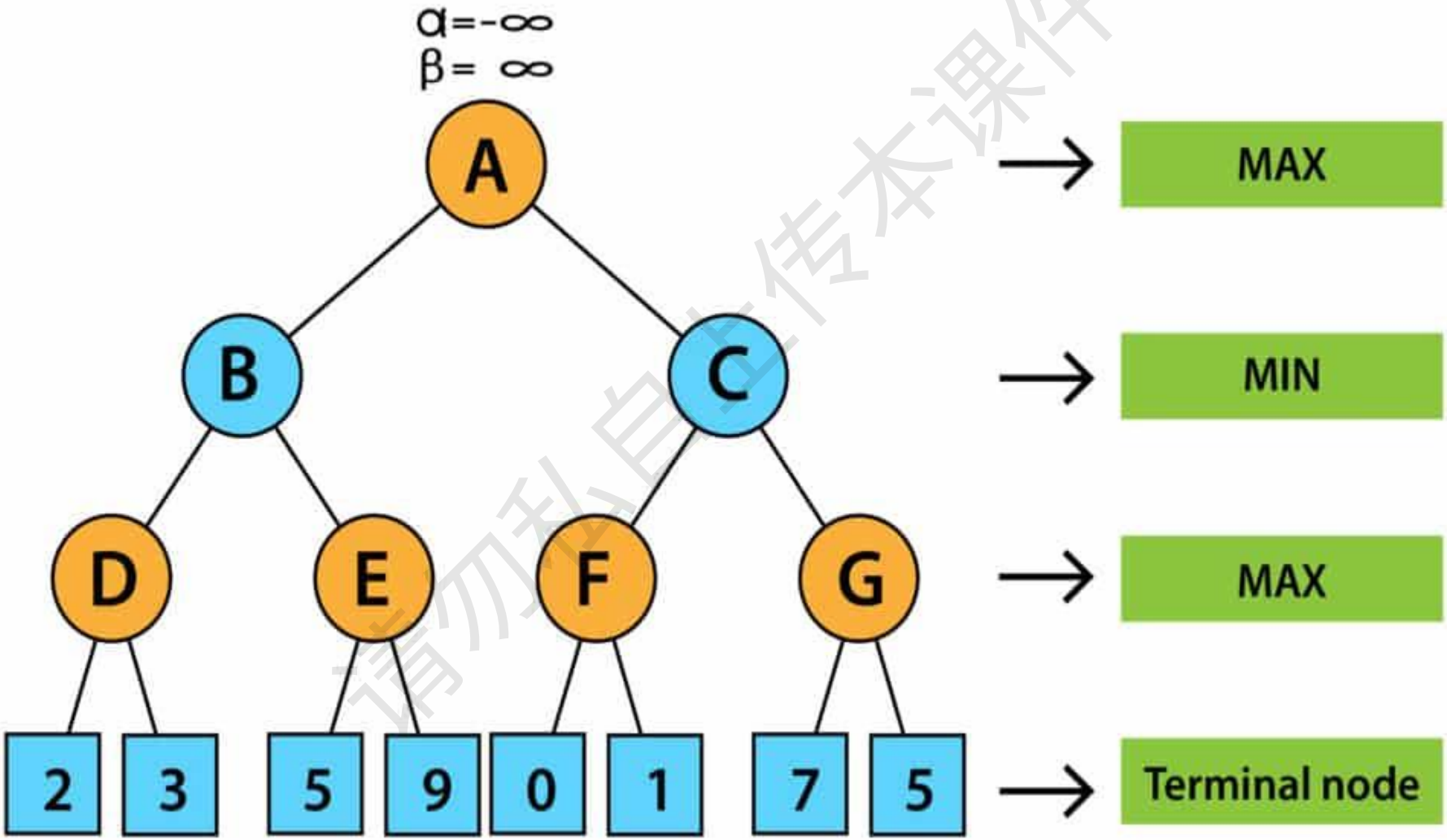
α=到目前为止，在 MAX 层找到的最佳（最大）选择值，即“我至少能获得多少”。为极大层的下界

β=到目前为止，在 MIN 层找到的最差（最小）选择值，即“对手最多让我得到多少”。为极小层的上界

请勿私自上传本课件

1.4 博弈树搜索

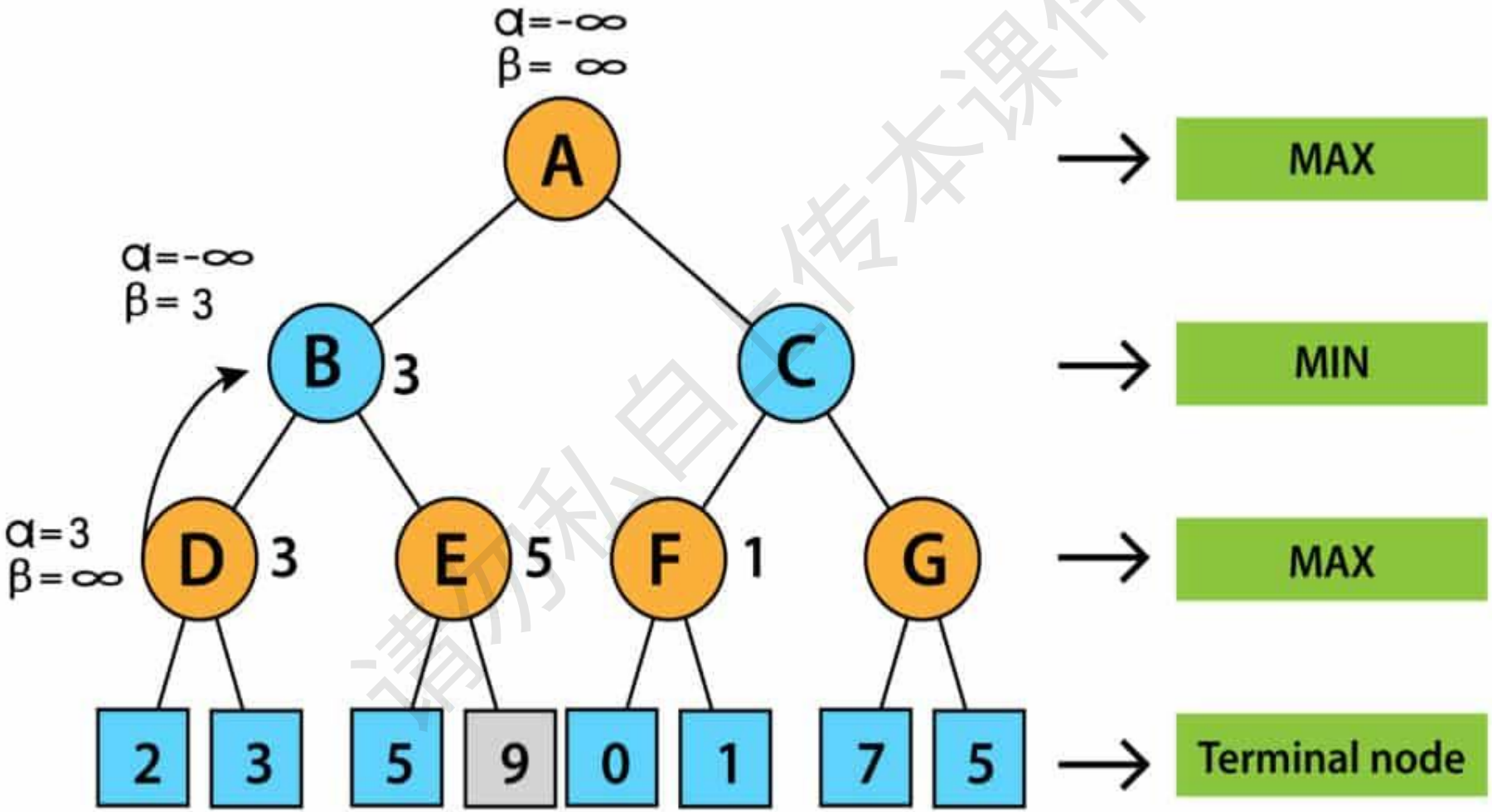
■ 博弈搜索　　α-β剪枝算法实例



请勿私自传本课件

1.4 博弈树搜索

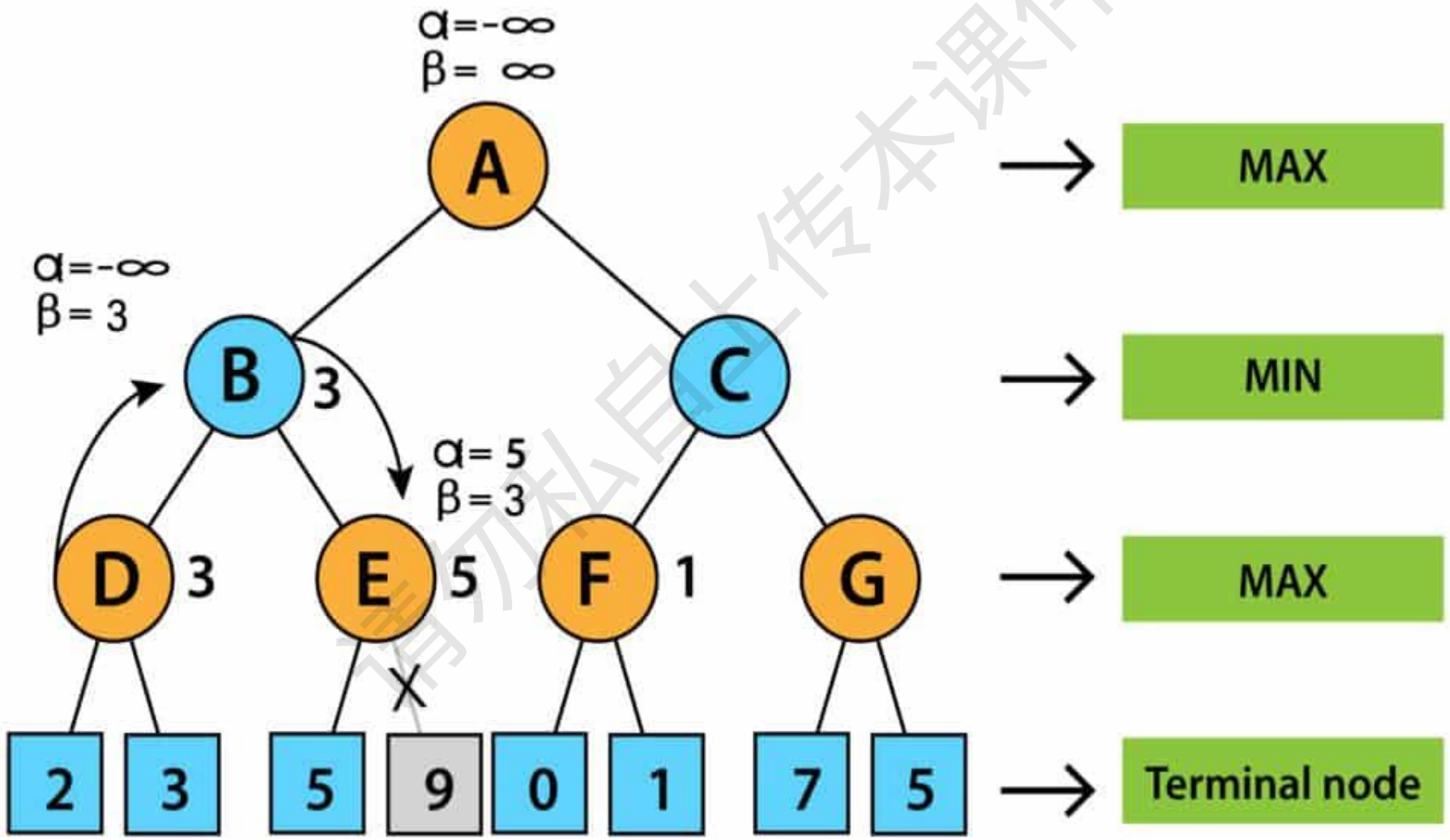
■ 博弈搜索　　α-β剪枝算法实例



请勿私自传本课件

1.4 博弈树搜索

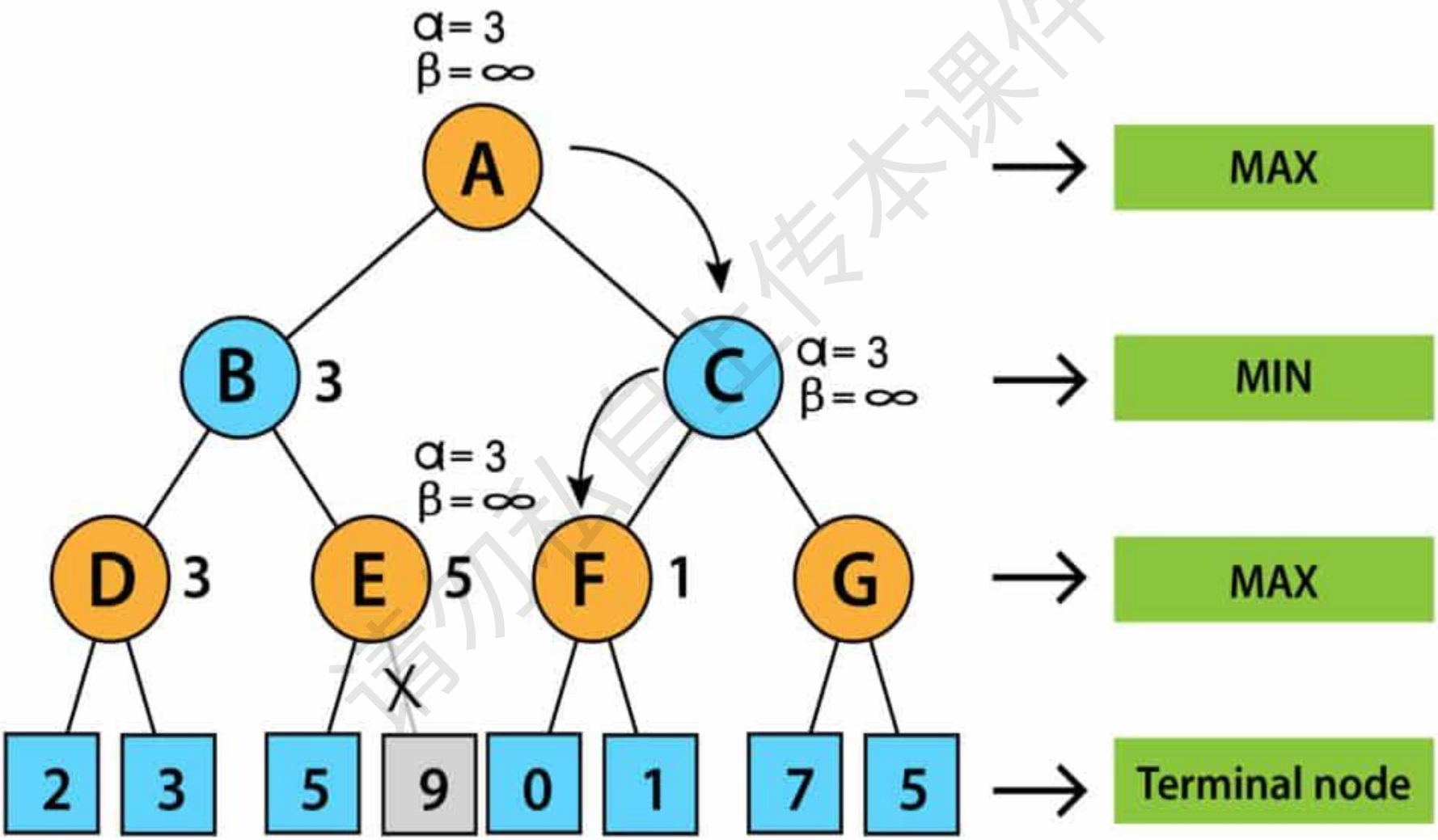
■ 博弈搜索　　α-β剪枝算法实例



请勿私自上传本课件

1.4 博弈树搜索

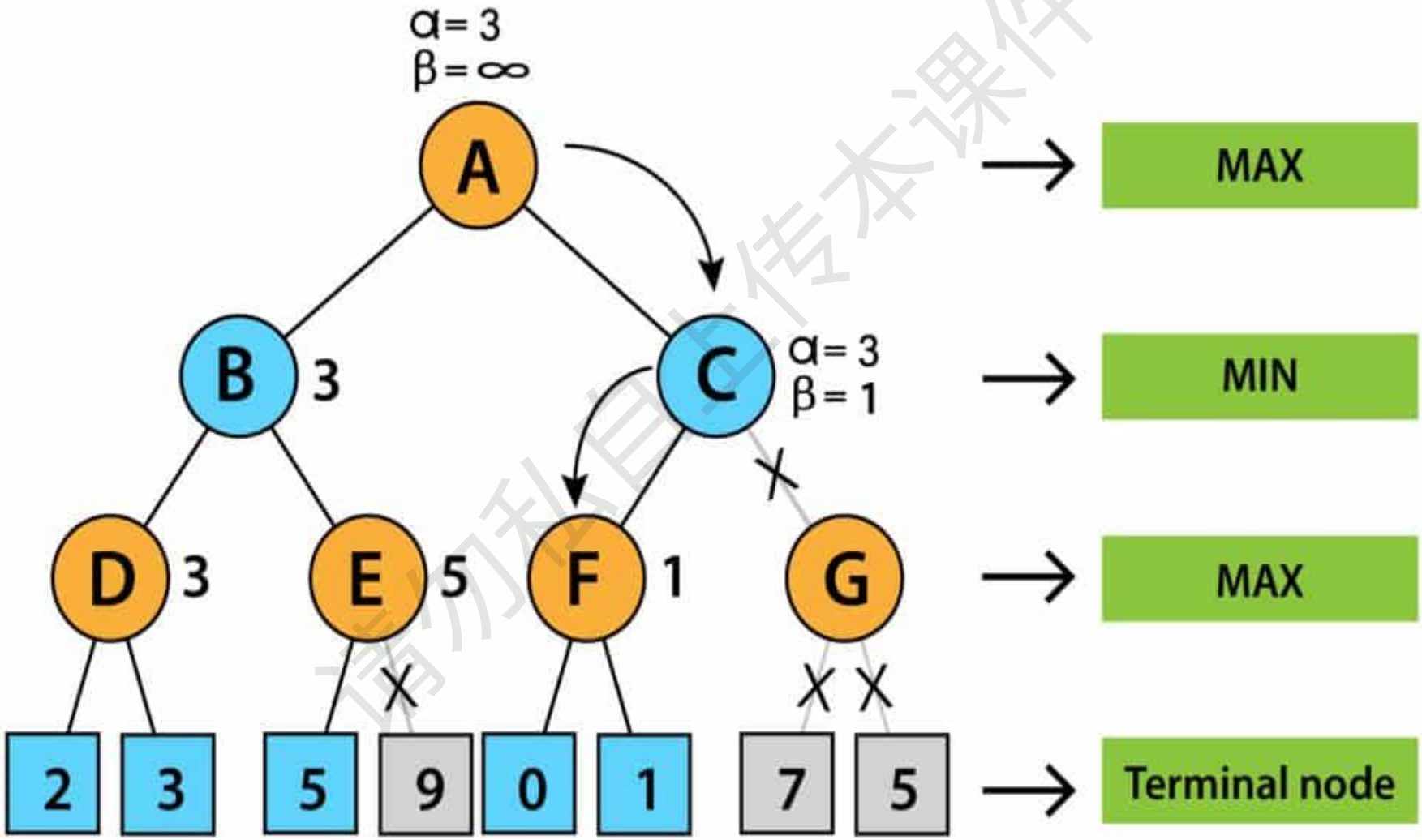
■ 博弈搜索　　α-β剪枝算法实例



请勿私自传播本课件

1.4 博弈树搜索

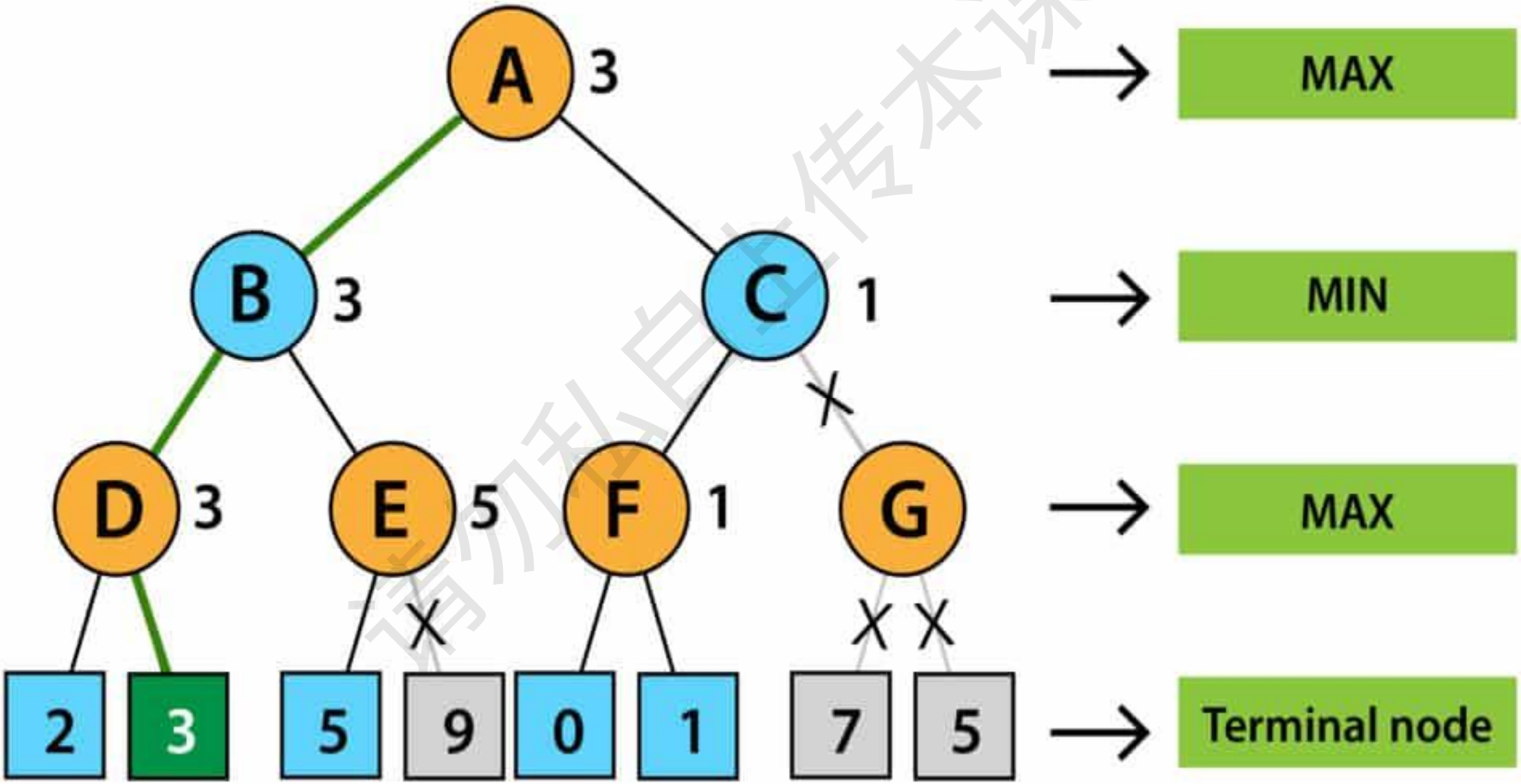
■ 博弈搜索　　α-β剪枝算法实例



请勿私自传播本课件

1.4 博弈树搜索

■ 博弈搜索　　α-β剪枝算法实例



请勿私自传本课件

1.4 博弈树搜索

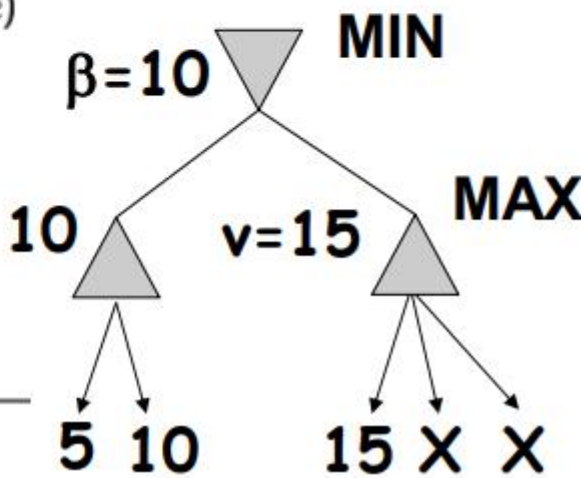
■ 博弈搜索　　α-β算法设计伪码描述

```
function ALPHA-BETA-SEARCH(state) returns an action
    inputs: state, current state in game
    v ← MAX-VALUE(state, −∞, +∞)
    return the action in SUCCESSORS(state) with value v

function MAX-VALUE(state, α, β) returns a utility value
    inputs: state, current state in game
            α, the value of the best alternative for MAX along the path to state
            β, the value of the best alternative for MIN along the path to state
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← −∞
    for s in SUCCESSORS(state) do
        v ← MAX(v, MIN-VALUE(s, α, β))
        if v ≥ β then return v        → Pruning
        α ← MAX(α, v)
    return v
```

New

β剪枝



请勿私自上传本课件

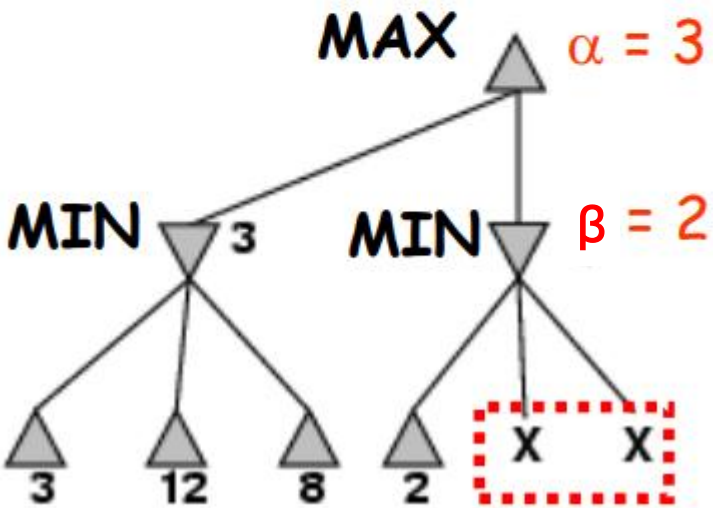
1.4 博弈树搜索

■ 博弈搜索　　α-β算法设计伪码描述

```
function MIN-VALUE(state, α, β) returns a utility value
    inputs: state, current state in game
            α, the value of the best alternative for MAX along the path to state
            β, the value of the best alternative for MIN along the path to state

    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← +∞
    for s in SUCCESSORS(state) do
        v ← MIN(v, MAX-VALUE(s, α, β))
        if v ≤ α then return v
        β ← MIN(β, v)
    return v
```

α剪枝 Pruning



请勿私自上传本课件